Structural Patterns: Proxy, Facade, Bridge, Decorator

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 12

Categorizing Design Patterns

- **Behavioural**

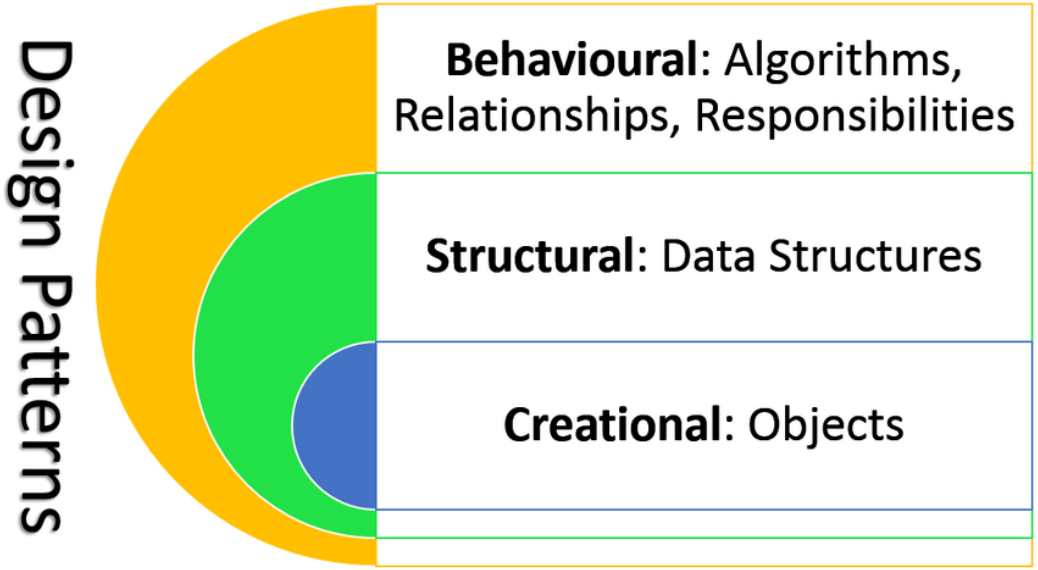
Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural** **(We are looking at these!)**

How do classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational**

All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects



Wrapper

What is a wrapper?

A class that **‘wraps’ around another class**.

The **wrapper** has the **same interface** as the ‘**wrappee**’.

Remember, this is interface in the OOP sense. The wrapper has the same public method and attributes as the wrappee. We can implement this using an ABC or by leaving it informal (duck typing).

All calls to the wrapper call the subsequent functions in the wrappee or the wrapped object.

What is a wrapper?

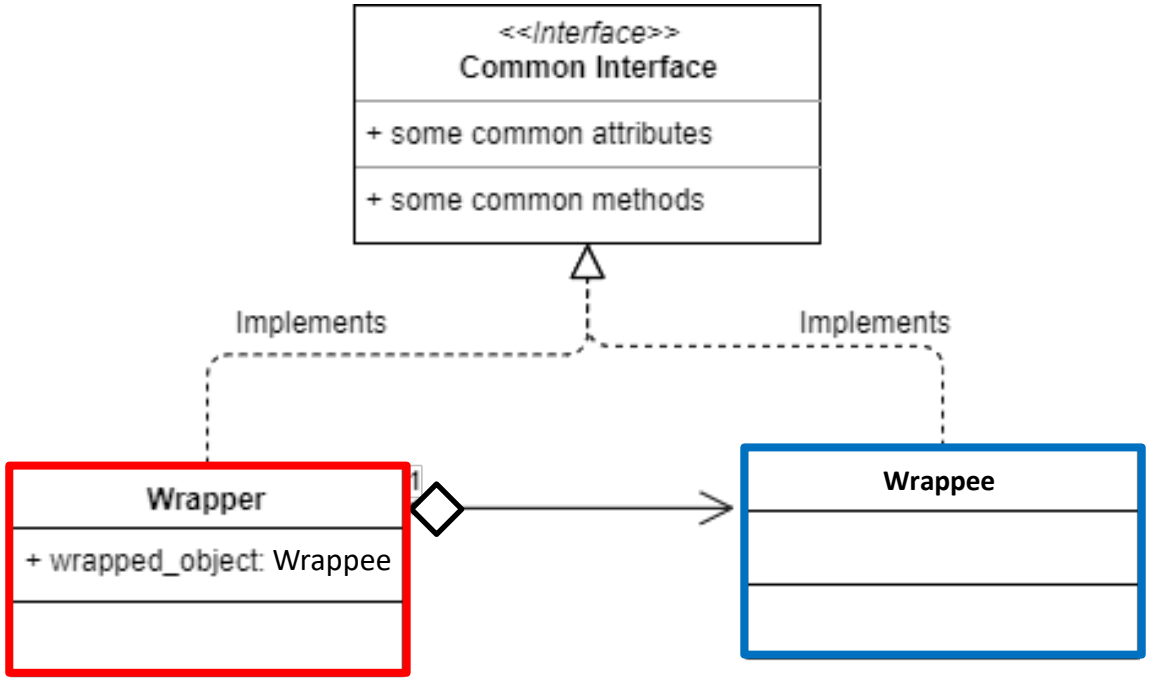
A **class** that **'wraps' around another class**. The **wrapper** uses a **has-a** relationship with **wrappee**.

In this example, the wrapper has the same public method and attributes as the wrappee. We can implement this using an interface or abstract base class

All calls to the wrapper call the subsequent functions in the wrappee or the wrapped object.

Wrapping an object, with another object of the same interface allows us the flexibility to perform extra processing before and after we make the call to the wrappee

Note – the wrapper does NOT always have to have the same interface as wrappee



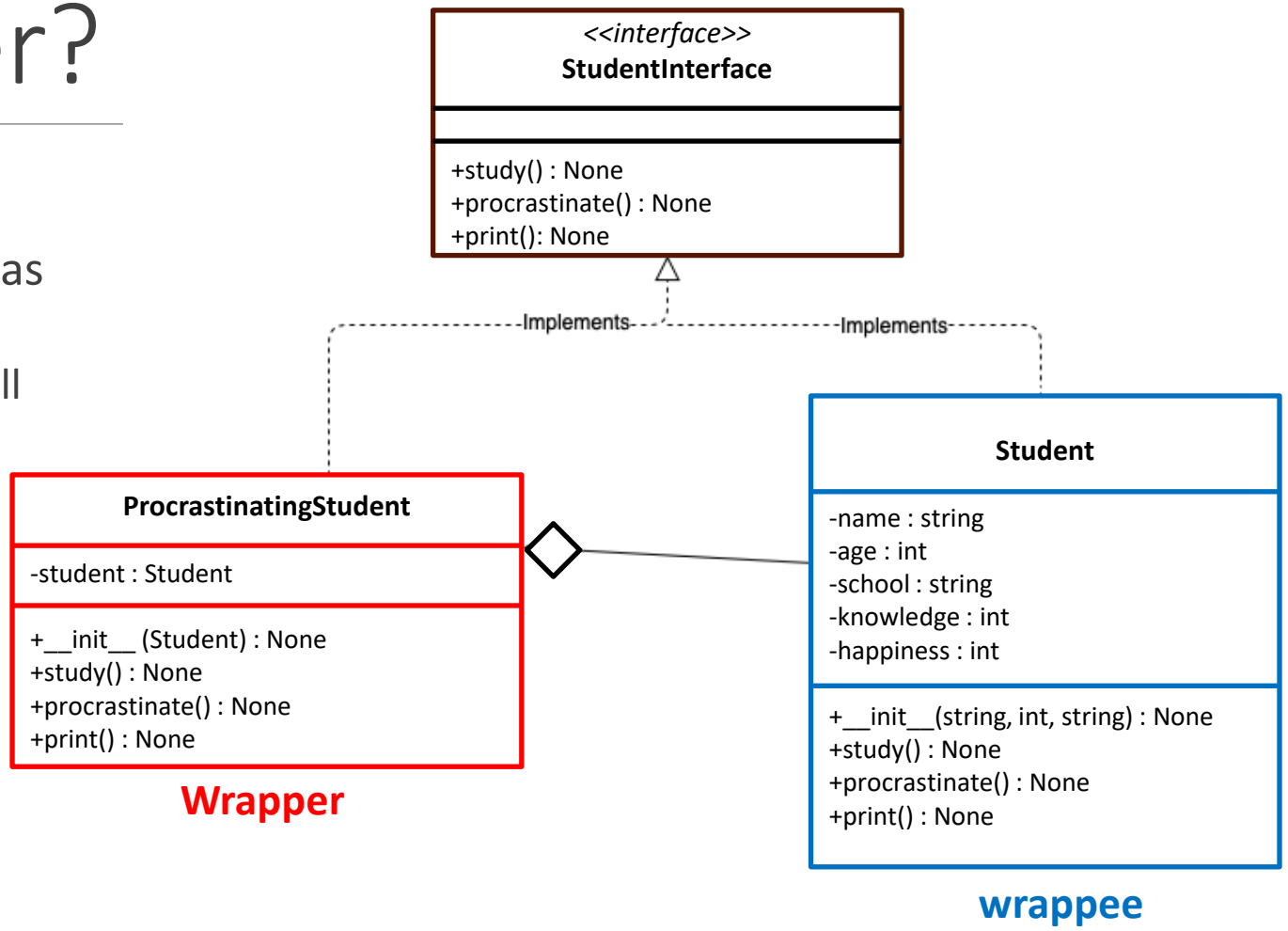
What is a wrapper?

Student is the wrappee

ProcrastinatingStudent is a wrapper. It has an instance of student

- Any calls to its methods will eventually call the student instance's methods, but with possibly additional behavior

Both classes implement the methods defined in the **StudentInterface**



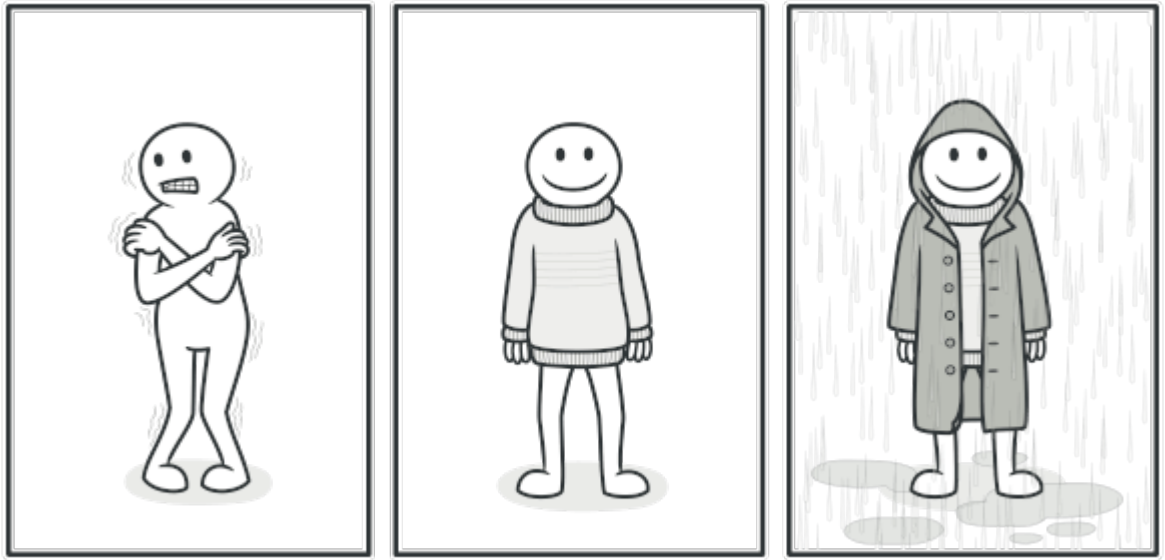
Wrapper.py

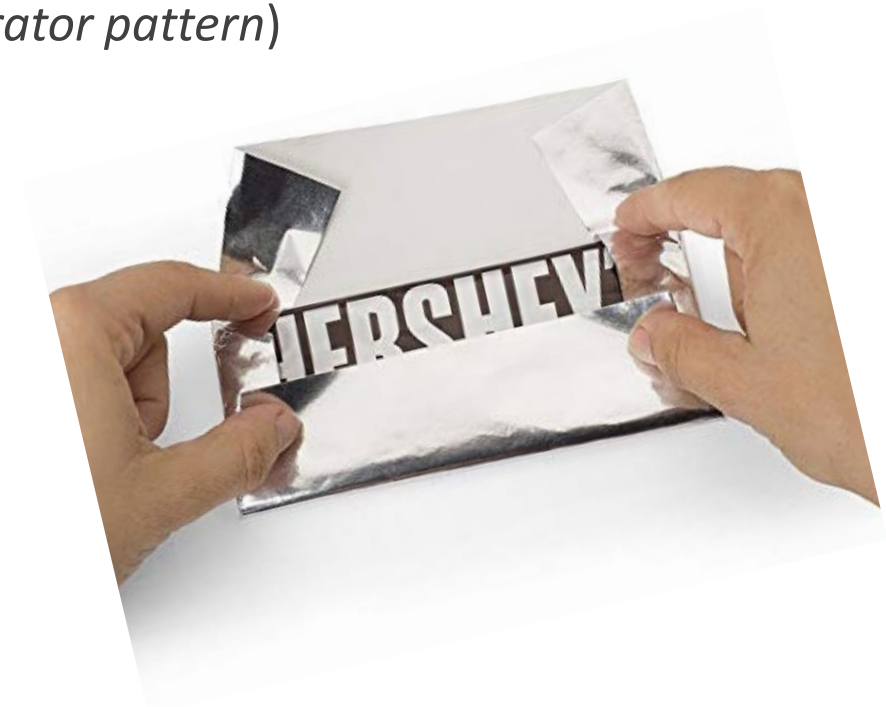
Wearing many layers (Multiple Wrappers)

We can wrap a base component in a wrapper to add some extra behaviour or functionality to it.

We can also wrap that wrapper in another wrapper to add more functionality

This can go on infinitely. (*We'll learn about this later…Decorator pattern*)





Proxy

Proxy

A Proxy is a wrapper.

It controls access to the wrappee.

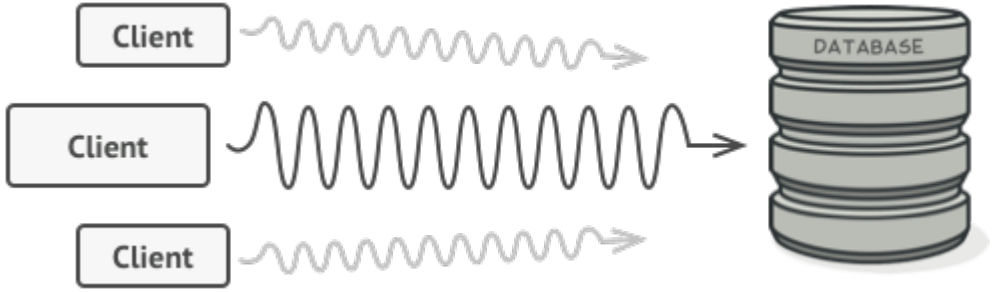
For example:

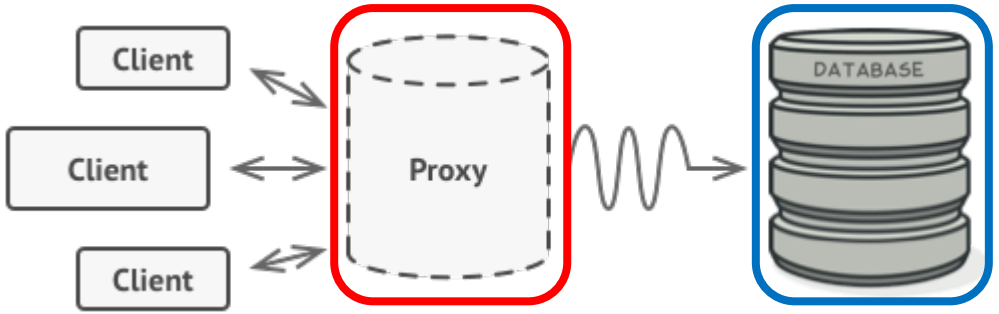
Database queries can be slow.

We might want to not only speed this up but also restrict access to the database. It's better to allow only objects with authorization.

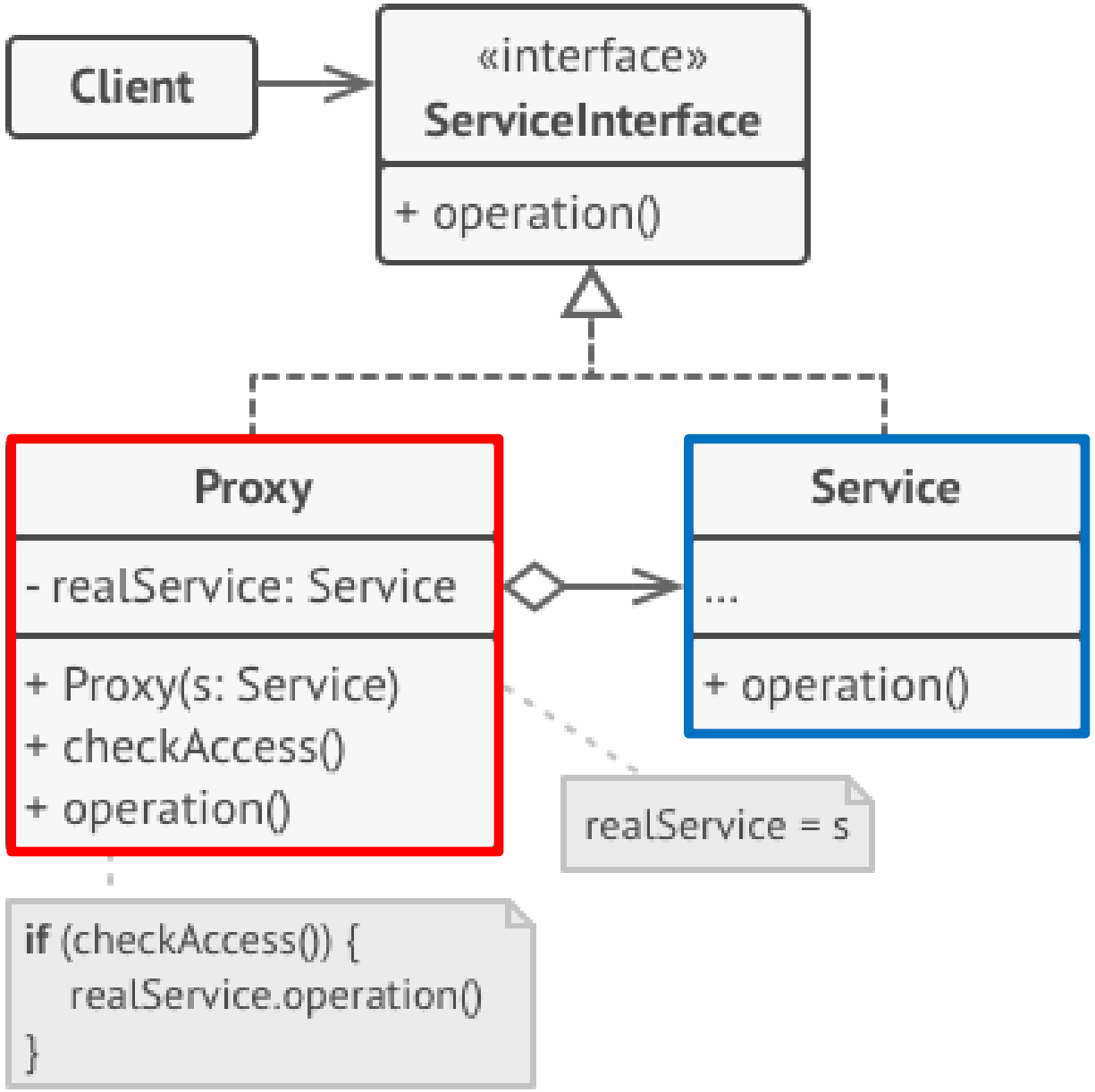
A **proxy** in this case, is an object that has the **same interface** as the **database** and has access to it.

The **proxy** can handle authorization, result caching and other optimizations. The client doesn't even need to know.

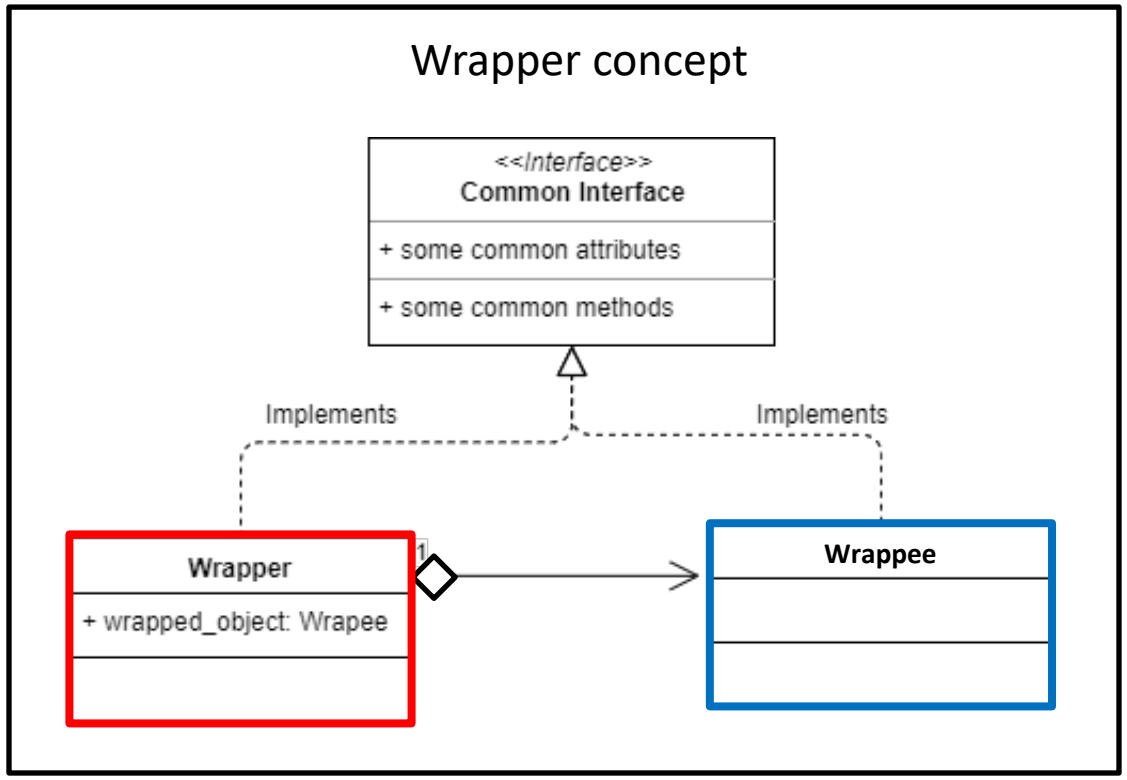


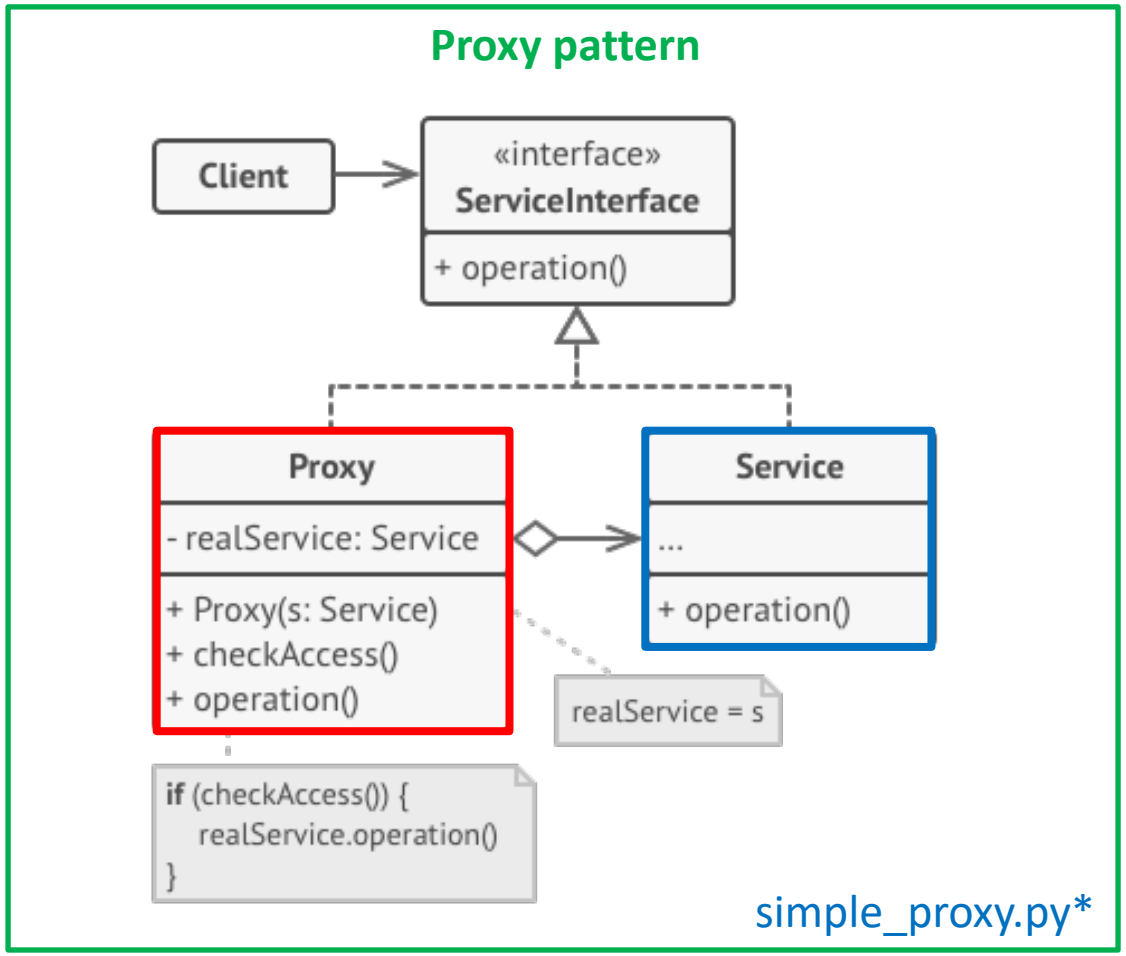


The Proxy Pattern



This is the exact same structure as a wrapper.





Let’s build a proxy

I want to create a movie database that can:

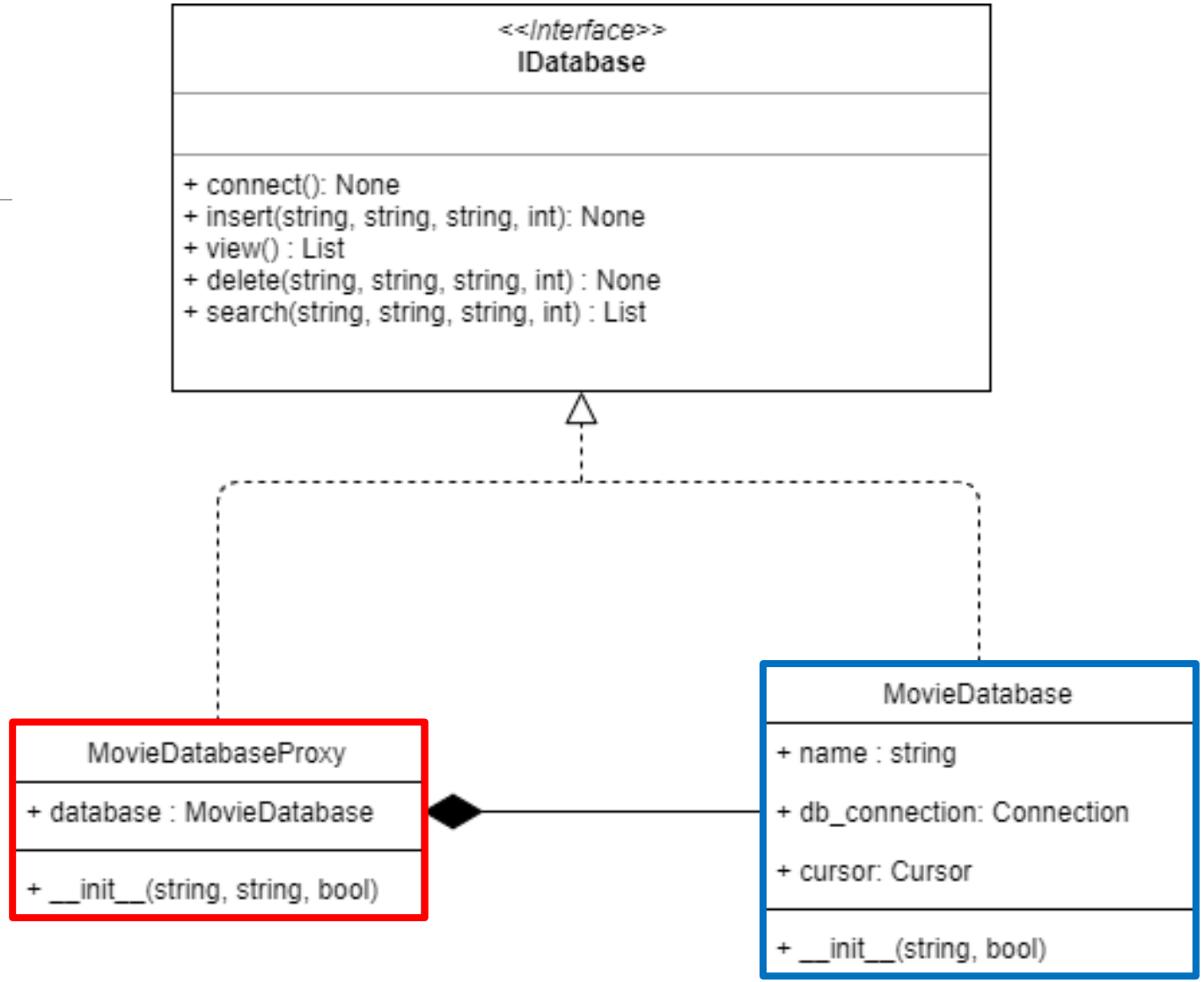
1. Insert movies
2. View movies
3. Delete movies
4. Search movies

But I want to **restrict access** to this database depending on type of user:

1. Moderator
2. Member
3. Guest

How can I do this without overcomplicating the MovieDatabase code?

Proxy with **formal interface**



Let's build a proxy

I want to create a movie database that can:

1. Insert movies
2. View movies
3. Delete movies
4. Search movies

But I want to **restrict access** to this database depending on type of user:

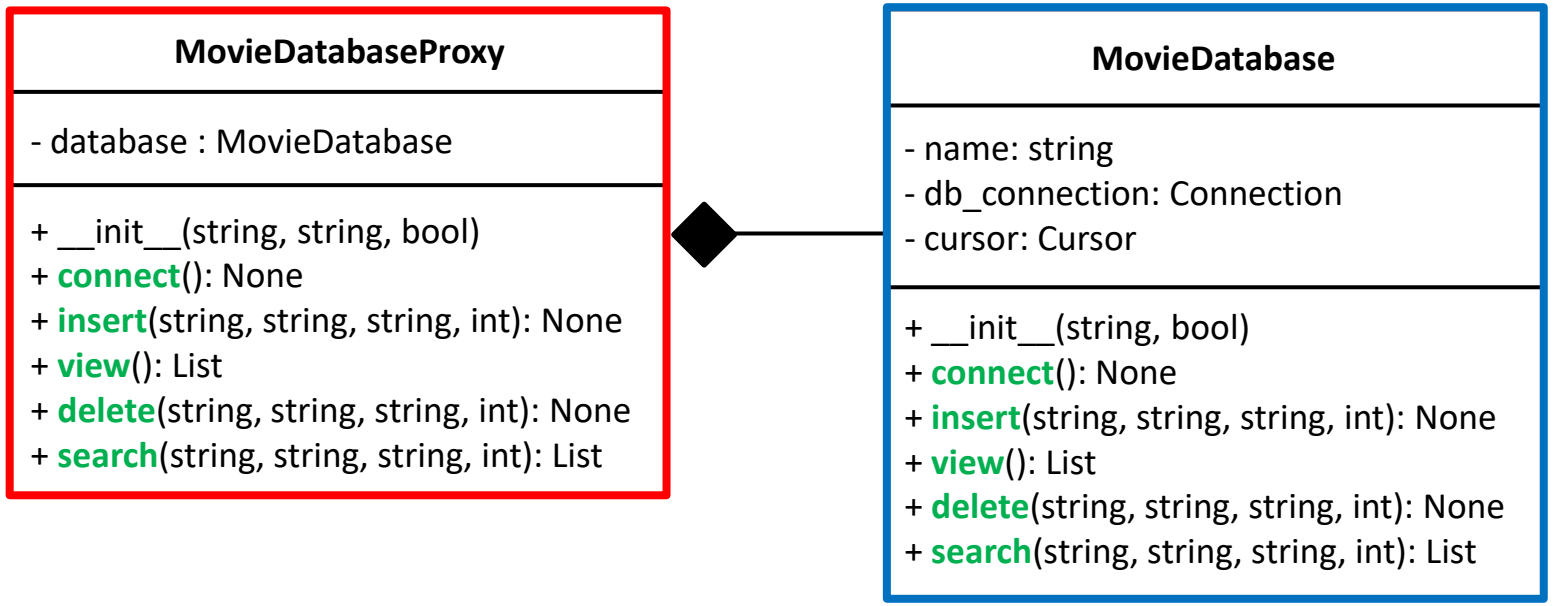
1. Moderator
2. Member
3. Guest

How can I do this without overcomplicating the MovieDatabase code?

movie_database_proxy_example.py

Same as previous slide, but using an **informal interface**.

Notice both classes contain the **same method names**



Proxy – Why and When do we use it

- If you want to execute something **before or after the primary service logic**
- Since the **proxy** implements the **same interface** as the **service**, it can be passed to any object that expects a **ServiceInterface**.
- Encapsulate and control access to **expensive objects**
- A Proxy can work even if the original service is not available.
- Follows the open closed principle. Can introduce **new proxies that extend behavior** instead of modifying the service.

Proxy – Disadvantages

- Can complicate the codebase if multiple proxies exist.
- Service **response might be delayed** if the proxy is carrying out extensive work before or after invoking the service.

Facade

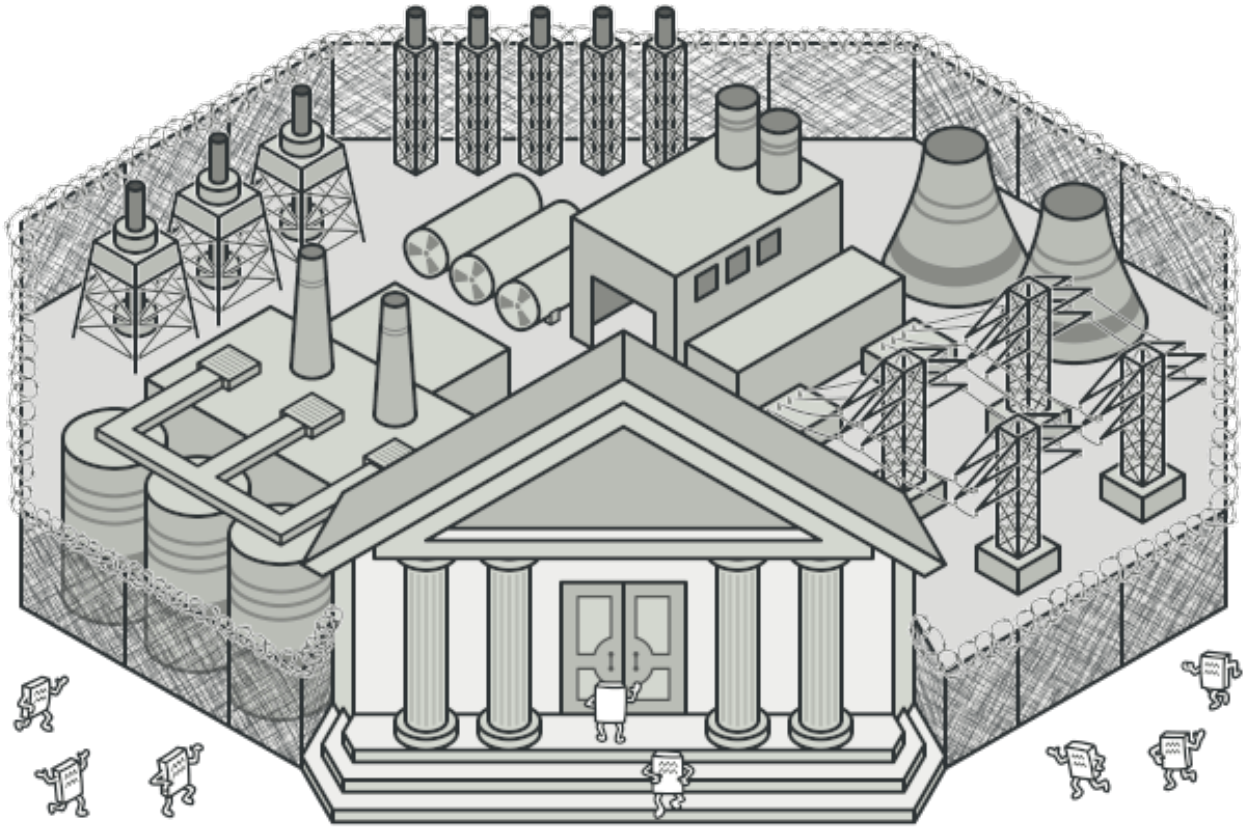
WHEN YOU WANT TO HIDE THE MESS RIGHT BEFORE YOUR FRIENDS COME OVER FOR BOARD GAMES…

Facade

The **Facade** pattern **encapsulates a complex system or API**

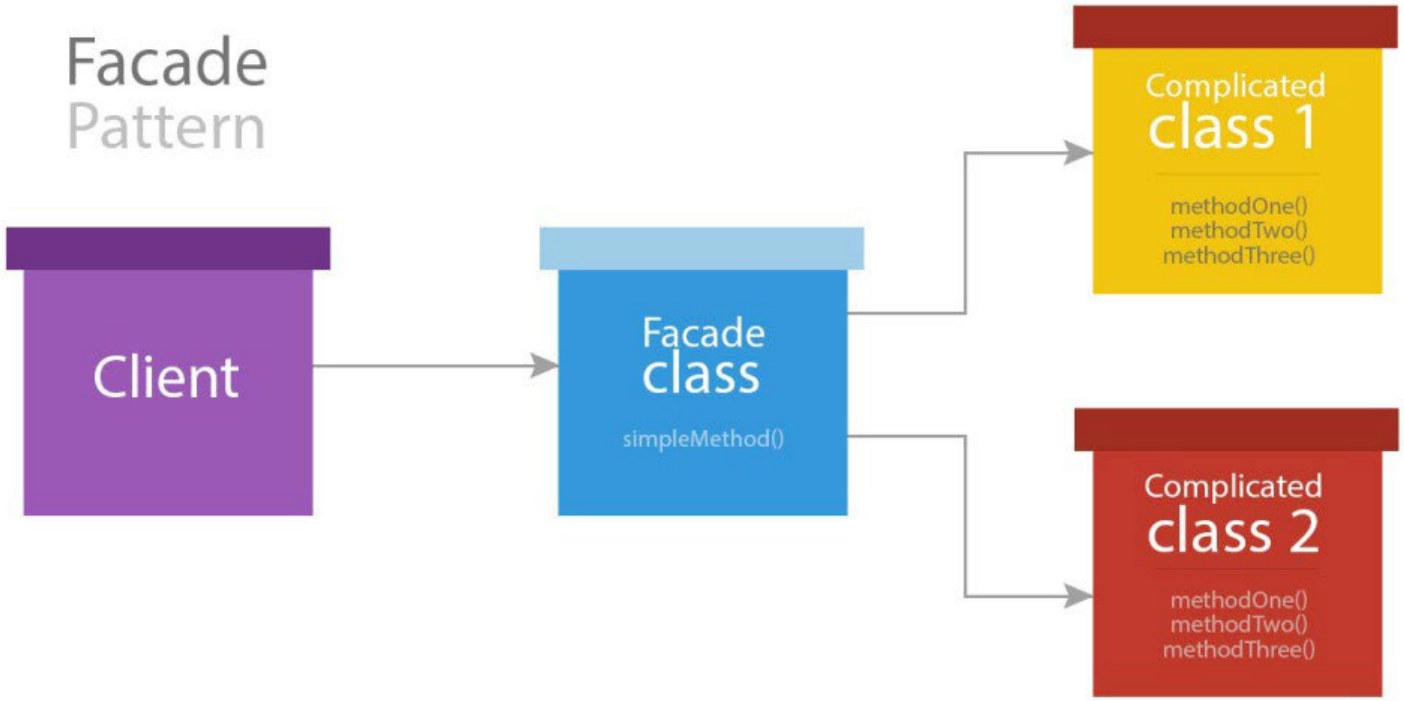
Used when we need to provide a **simple interface to a complex system**

Abstracts away the details of the complex system or API

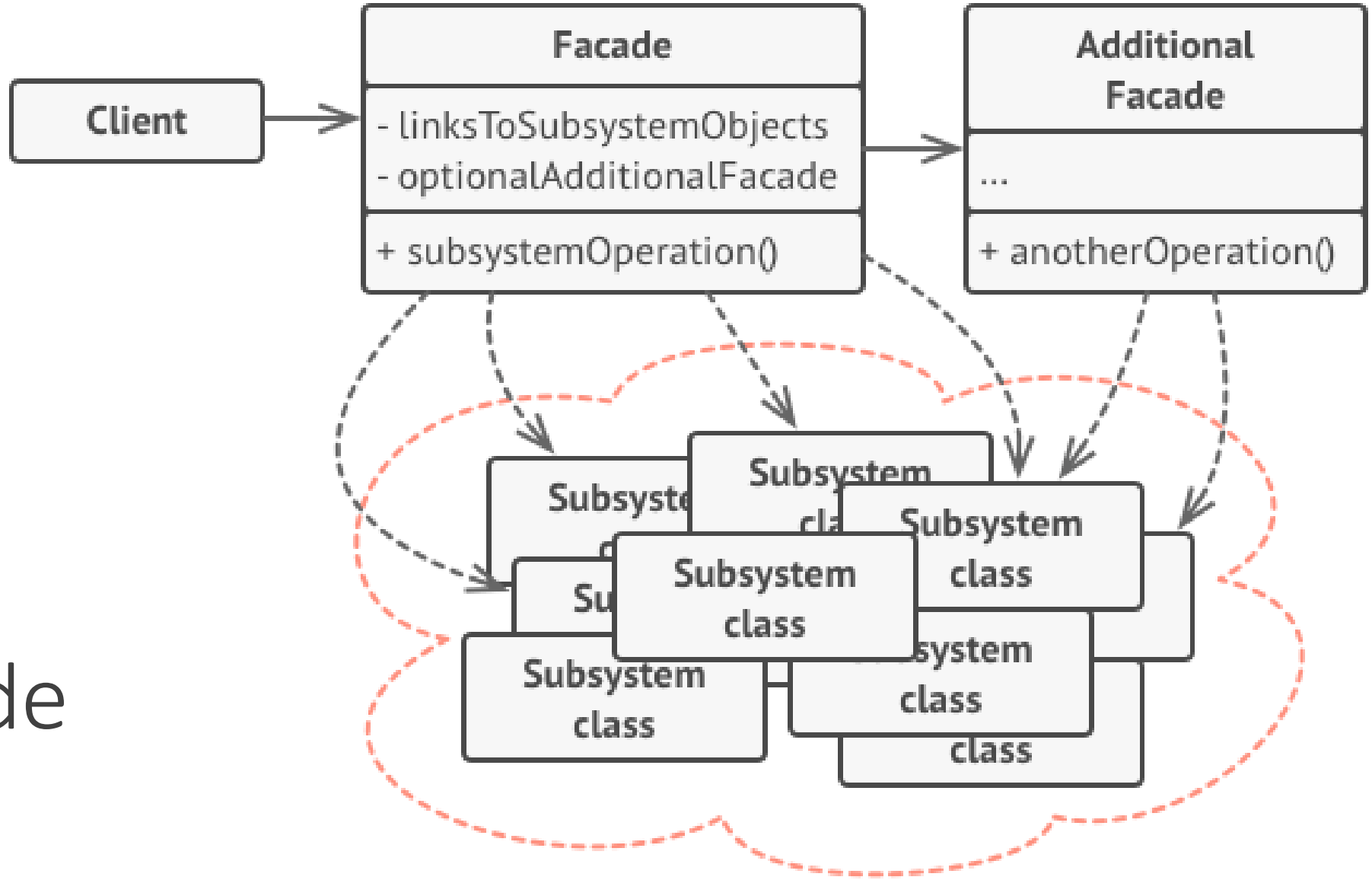


Facade

- Often we need to implement a complex API in our program
- This complex API / system has a lot of classes and moving parts
- Often we import such packages to only use a subset of those features and to solve a specific need.
- Façade encapsulates these complex system and hides their complex behaviour.
- The rest of our code only needs to be aware of the **simple interface** provided by the **Façade**.



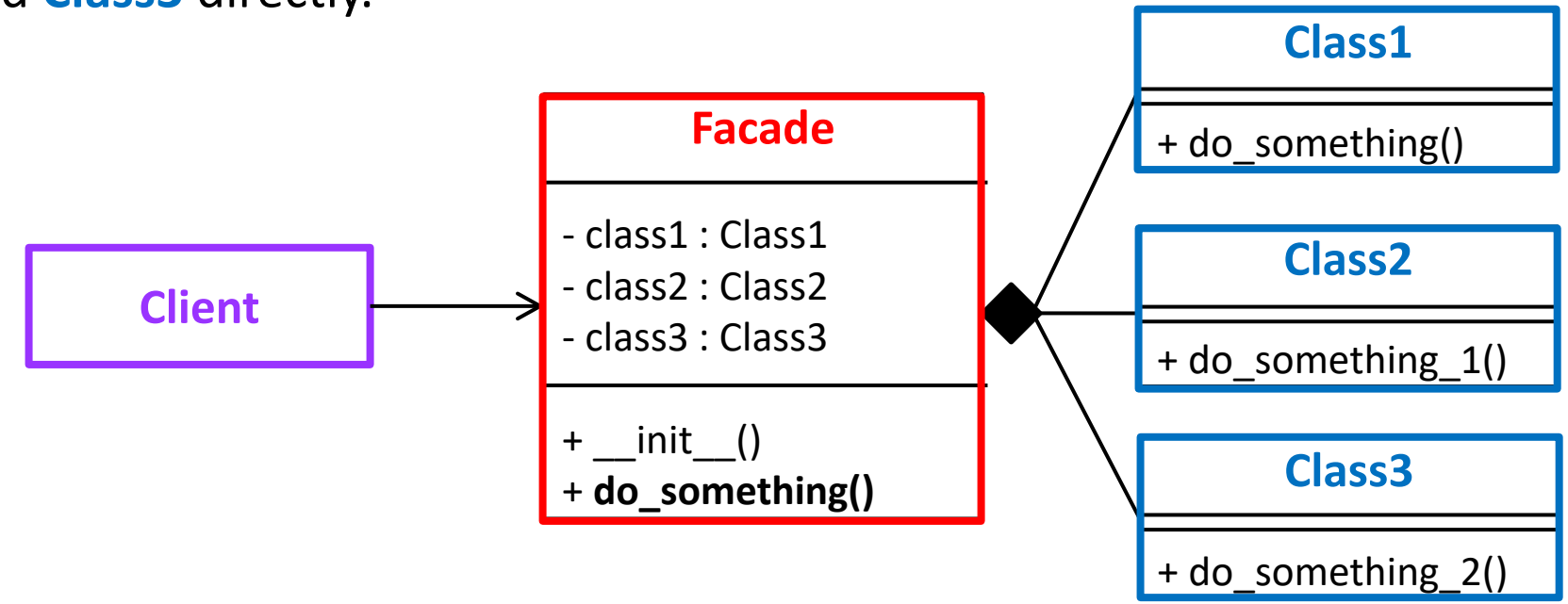
The Facade Pattern



Simple Facade

- **Problem:** **Client** wants to perform an action that only a sequence of calls to **Class1**, **Class2**, and **Class3** will complete. Don't want to couple **Client** with the **classes**

- Hide the sequence of calls in **Façade**'s **do_something** method
- **Client** now only needs to call **Façade**'s **do_something** method instead of calling **Class1**, **Class2**, and **Class3** directly.



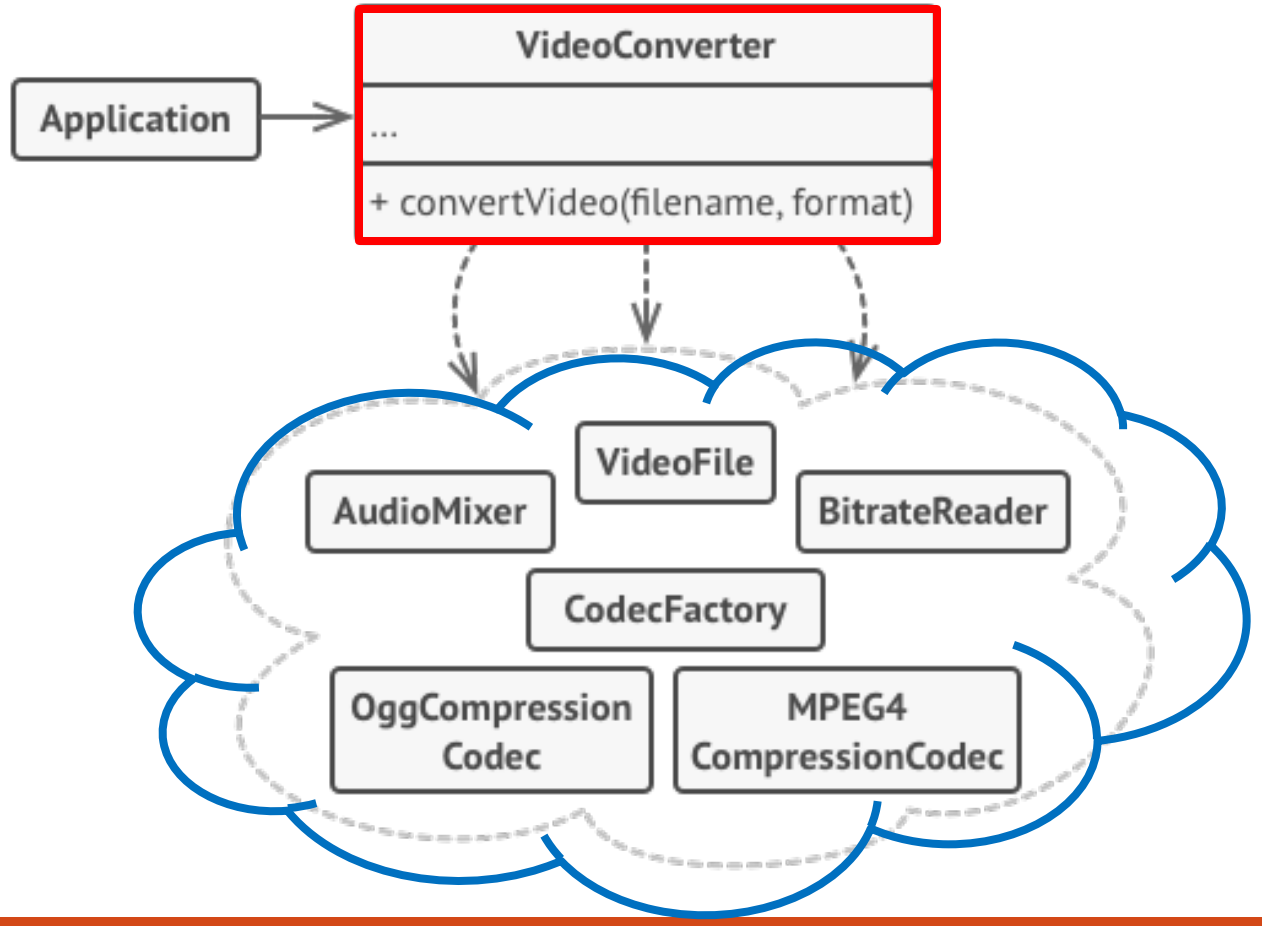
simple_facade_py

Facade Example

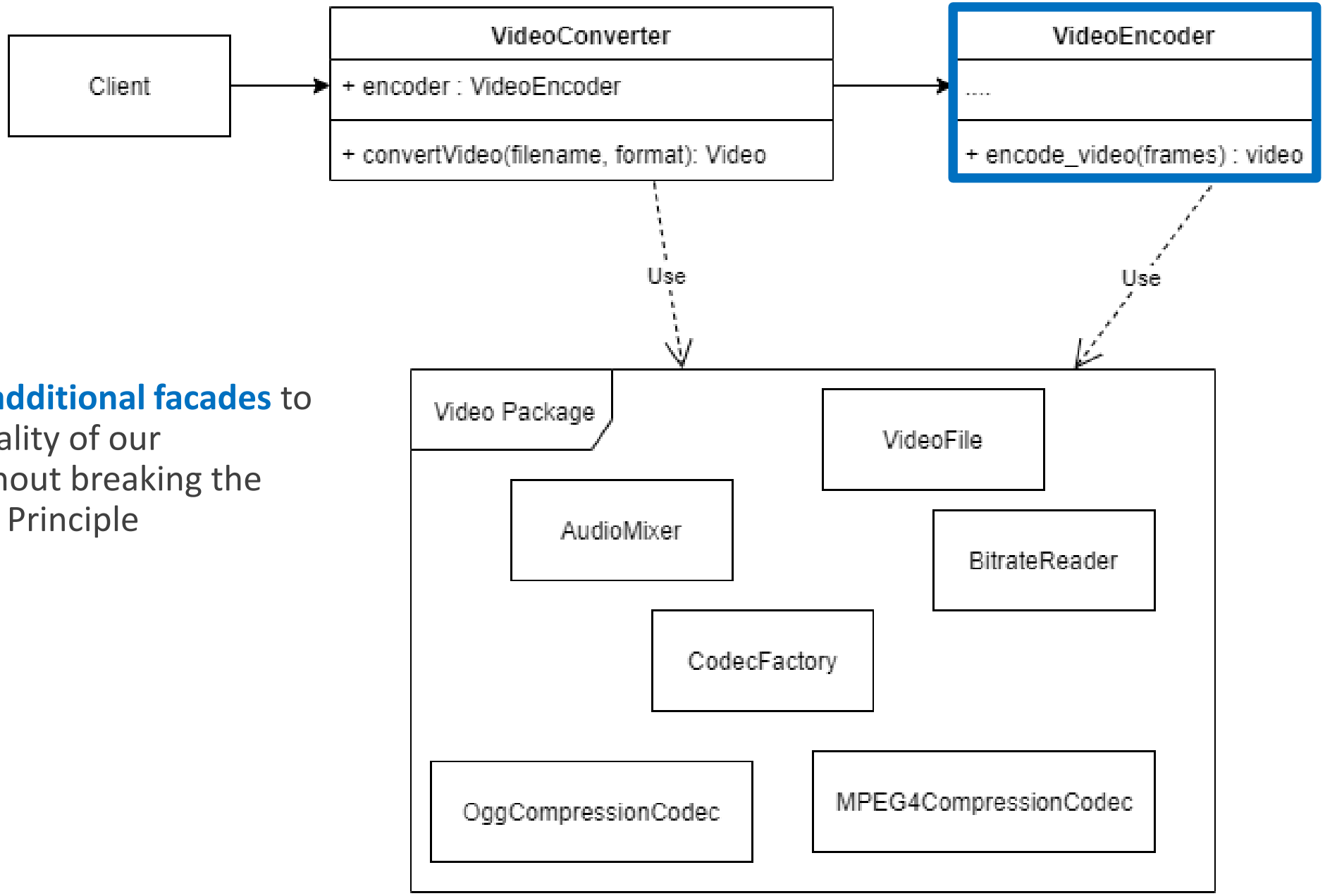
Say you imported a video conversion package that offered different ways to convert a video file with a number of customizations.

We can create our own **VideoConverter** file that accesses and uses this package while providing our system with a **simple interface** and **hiding the complexity**.

video_package_facade_example.py



We could even add **additional facades** to expand the functionality of our **VideoConverter** without breaking the Single Responsibility Principle



Facade – Why and When do we use it

- Good use of **Encapsulation** and **Data Hiding**
- When we want to **avoid complex architectures** if different parts of your code were dependent on complex tools / libraries / sub-systems
- If we only want to use a **small part** of a **larger more complex system**.
- Can be used to organize a system into layers.

Facade - Disadvantages

- A Facade can become an **extremely complex and large class** to maintain.
- It can become an **epicentre of coupling**

Bridge

WHEN COMPOSITION SAVES THE DAY

Bridge

A pattern that lets us split a large class, or a set of closely related classes into separate hierarchies.

The class can usually be split into ***"Abstractions"*** and ***"Implementations"***.

Yes. I *Italicized,* **bolded**, increased the sizes and put quotation marks around those words.

No. I am not a monster for doing that**. <u>Those words mean something a little different</u>** when it comes to the Bridge Pattern.

Let's look at a problem.

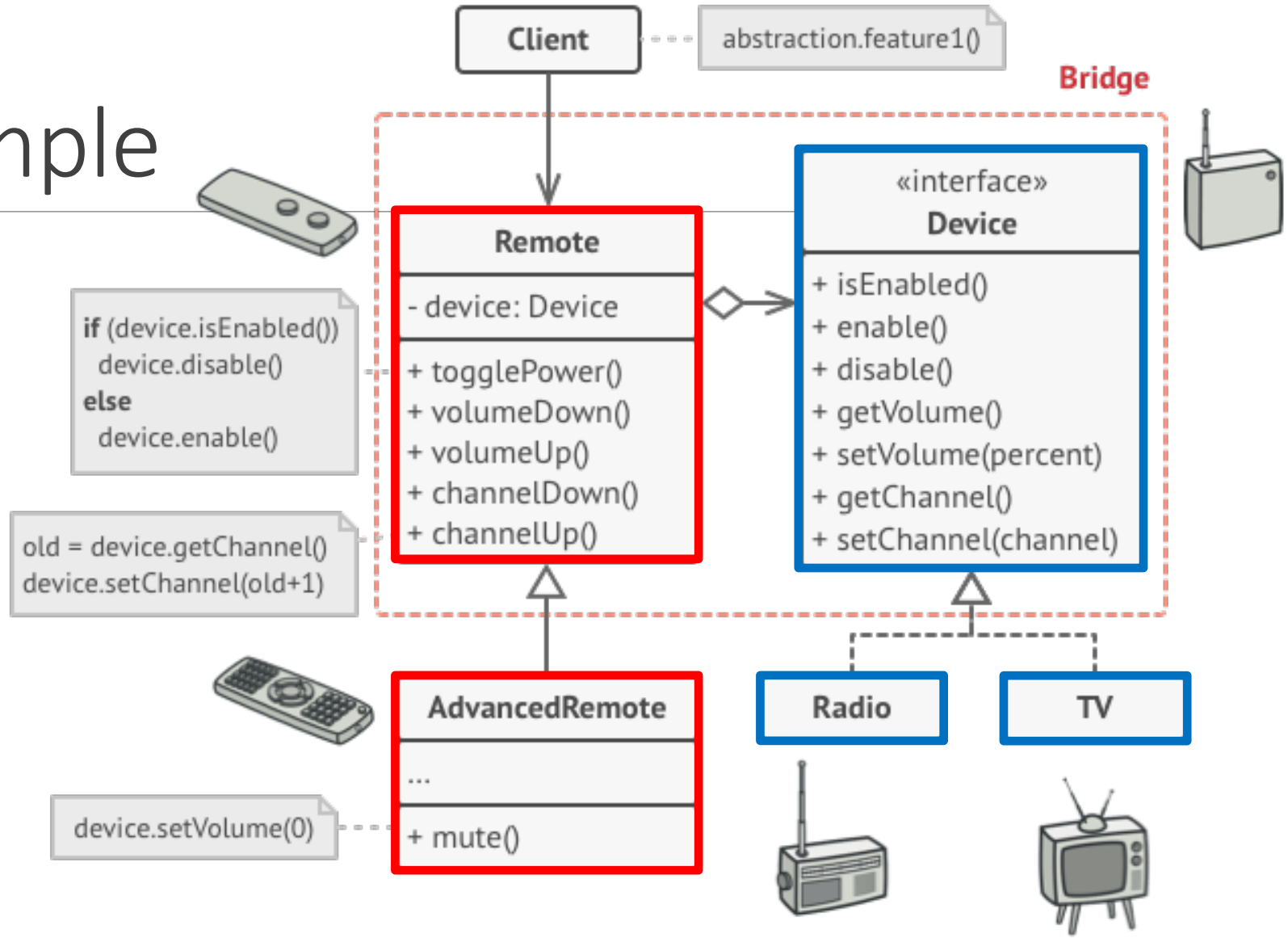
Bridge – A different example

A **Remote** is an **abstraction** of a device.

The **Remote** doesn't do anything by itself. It just acts like a **control layer** controlling some device implementation.

The Bridge Pattern lets us manage two sets of classes that define an entity where one dimension of it (Remote) is coupled with another dimension (Device).

Let's implement this example.



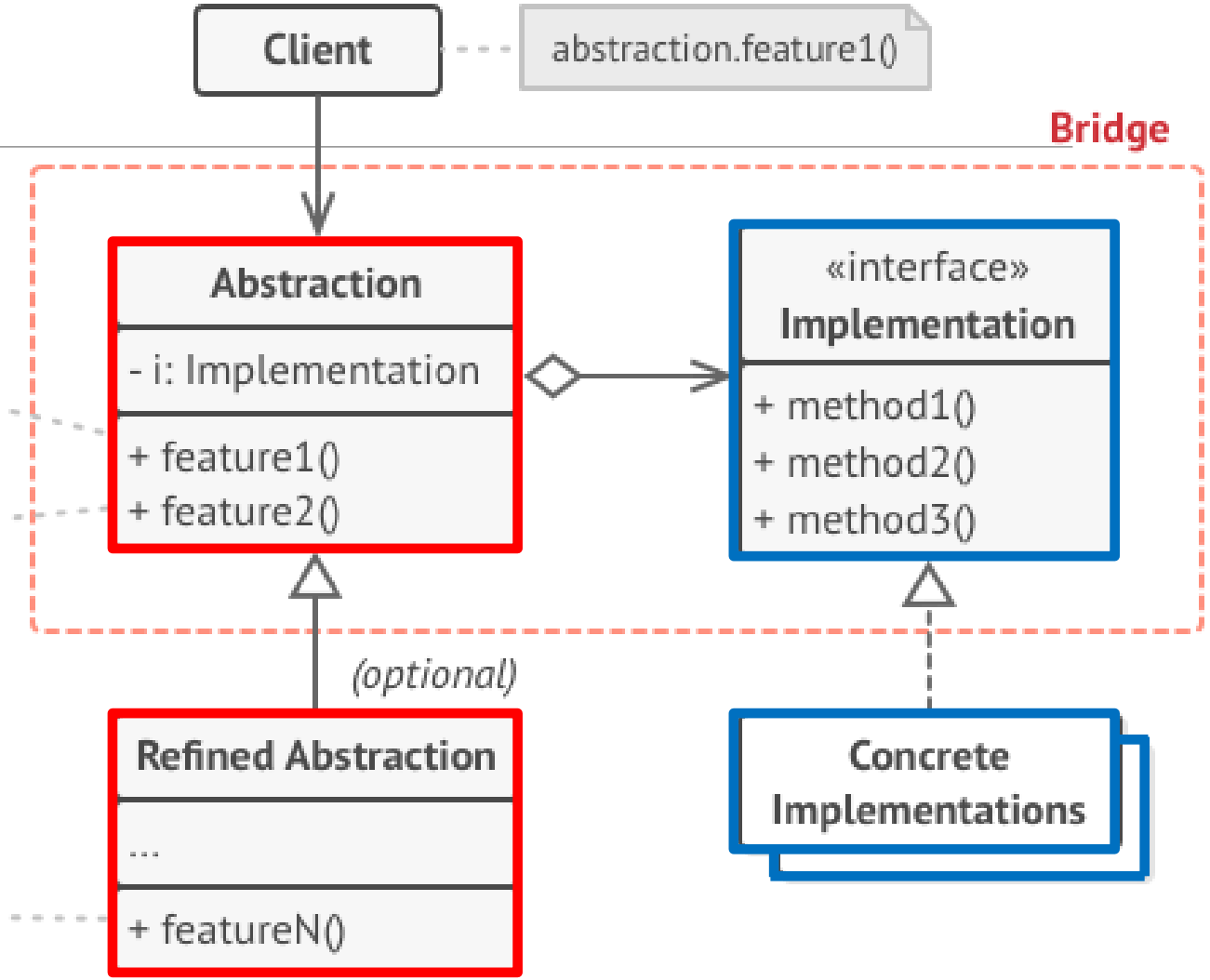
device_remote_bridge_example.py

Bridge

This is what the bridge pattern's UML class diagram looks like.

We have **2 hierarchies**. The **Abstraction hierarchy** and an **Implementation Interface hierarchy**.

We say that the **Abstractions** control the **Implementation**. The Remote just controls the Device. It doesn't do anything else.



Bridge – When do we use it and Why?

- When we want to **divide** and conquer a **huge class** that has many variations.
- This pattern allows us to manage entities that have multiple orthogonal dimensions that can each be extended separately independent of each other.
- We can use the Bridge Pattern if we want to **switch implementations at run-time**. For example. if we want to provide a DataSource a different input/output stream.
- Implements the Open/Closed Principle. Introduce new abstraction and implementations independently.
- Actually it manifests all the SOLID principles.

Bridge – Disadvantages

- Sometimes the code can get really complex if we have a highly cohesive class where the **different dimensions are not really independent** of each other.
- The idea of what our "abstraction" and our "implementation" is can get vague and artificial sometimes. Making the design difficult to understand.

Decorator Pattern

The problem

How do I **support multiple different 'optional' behaviours** without inflating my inheritance hierarchy and still maintaining the Open/Closed Principle?

An Example:

Donut shop menu

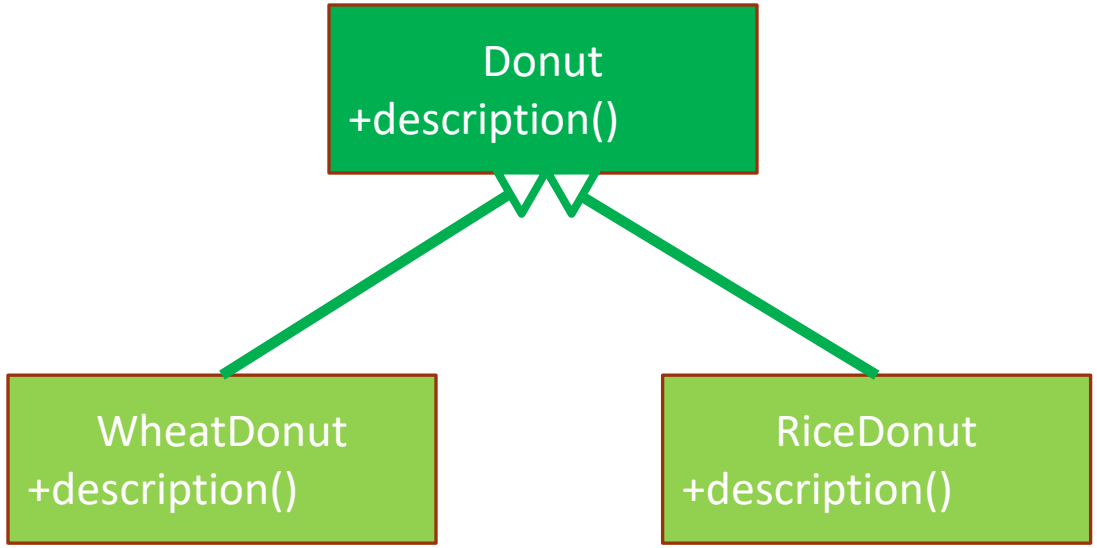
A different class for each donut, they all share the same base class and interface but come in infinite combinations and varieties.

You sell two different types of donuts

- Wheat donut
- Rice donut

Introduction - Problem

- You might model it to look something like this

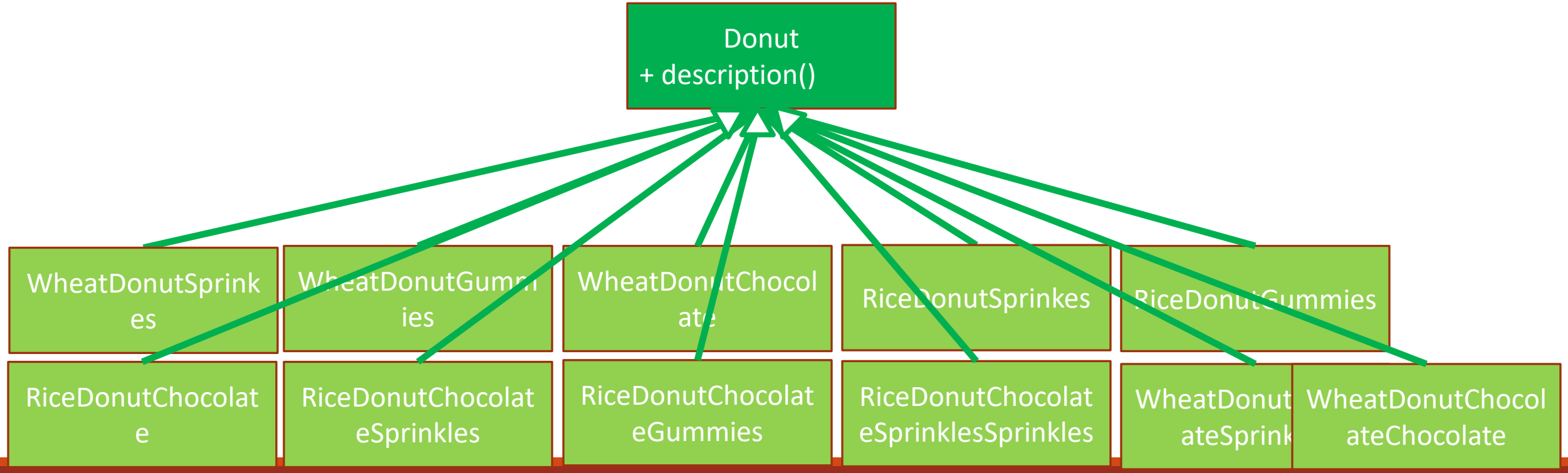


Introduction - Problem

- But then I ask you to have multiple toppings
  - Sprinkles
  - Gummies
  - Chocolate

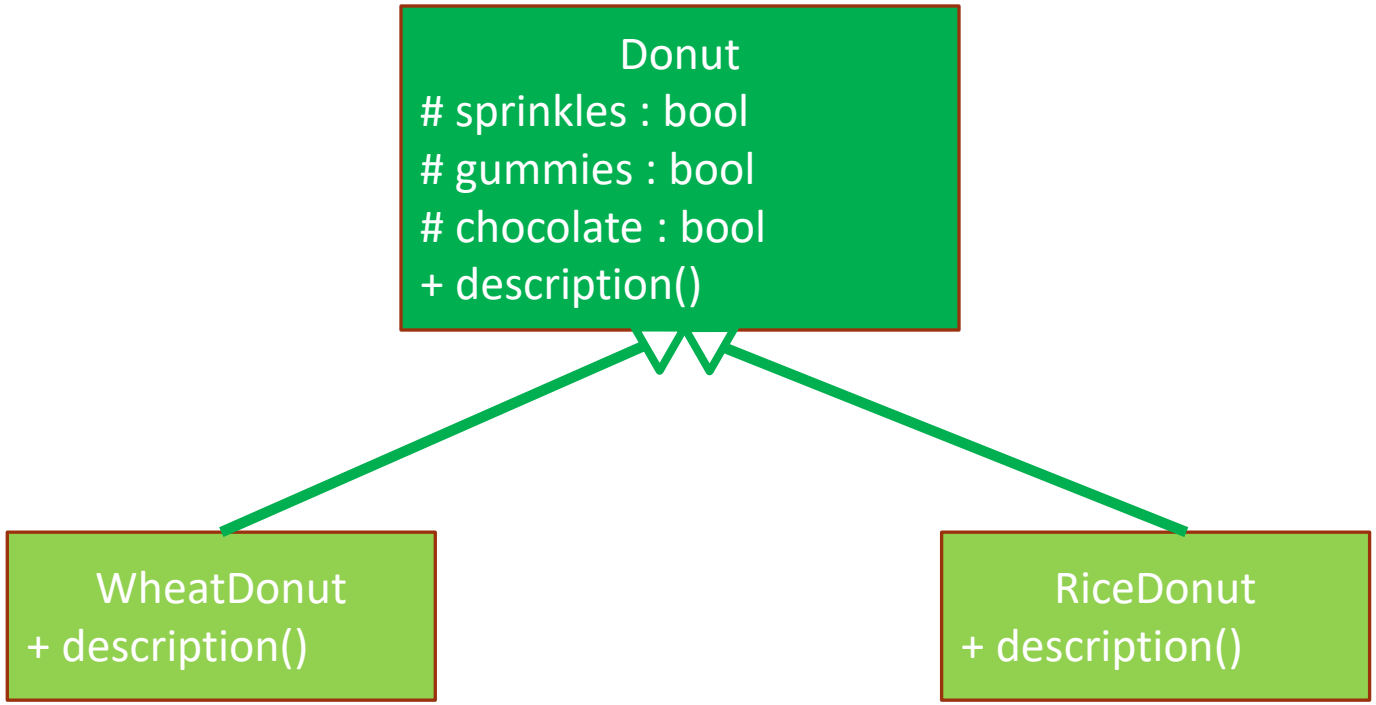
Introduction - Problem

- You might model it to look something like this (Please don't…)
- Imagine if I asked for combinations (Rice Donut Sprinkles Chocolate)



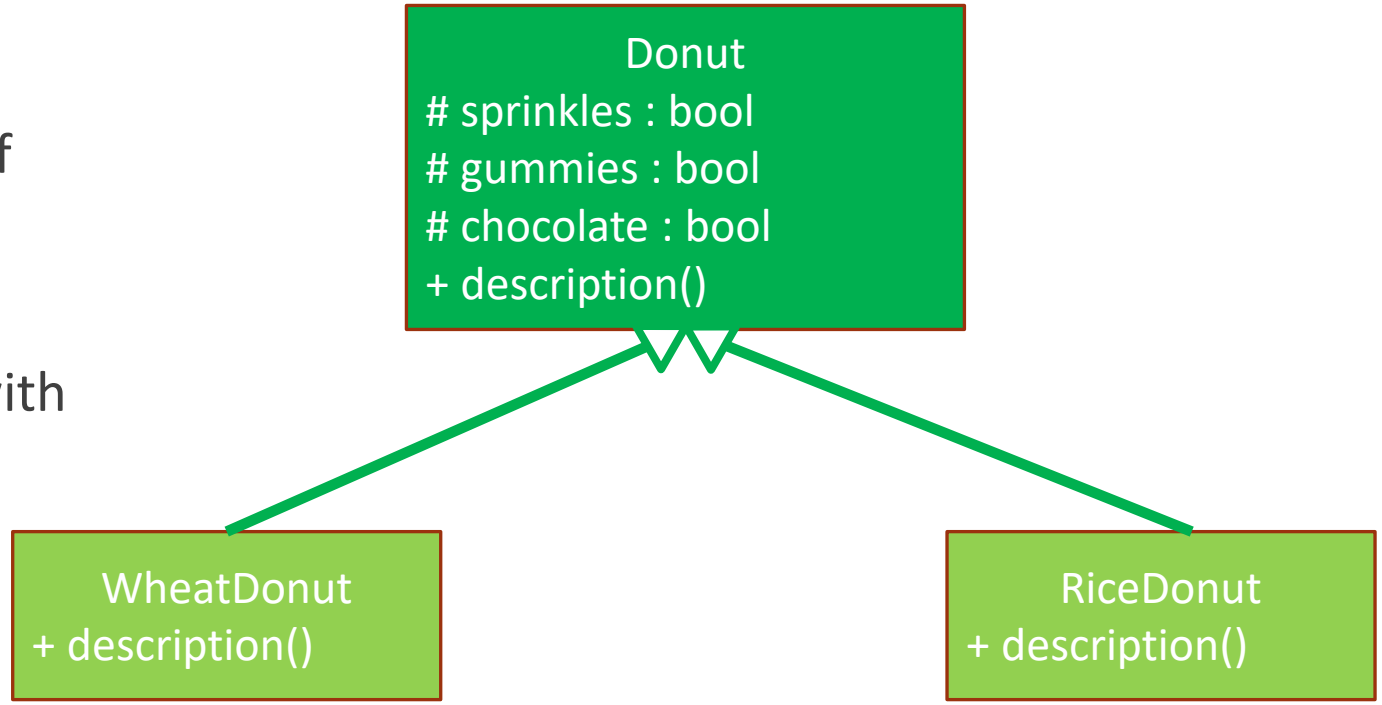
Introduction - Problem

- We can move the toppings to the Donut instead



Introduction - Problem

- Issues
  - Violates Open-Closed Principle
  - Will need to modify parent Donut if
    - New toppings added
    - Have double toppings
  - Some toppings might not go well with some Donut, but still inherited



Introduction - Decorator

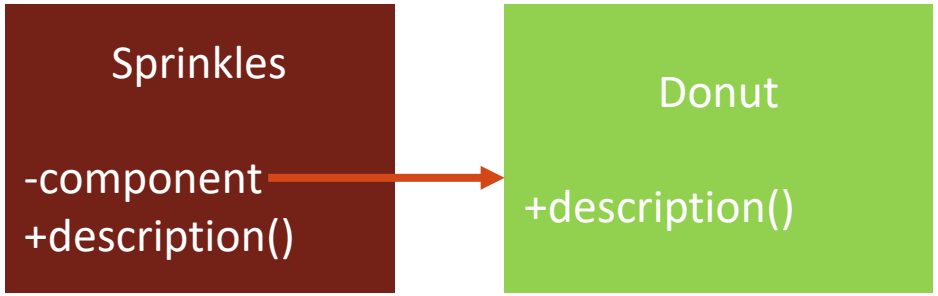
- Let’s try something different
- Let’s start with a **donut** object

Donut

+description()

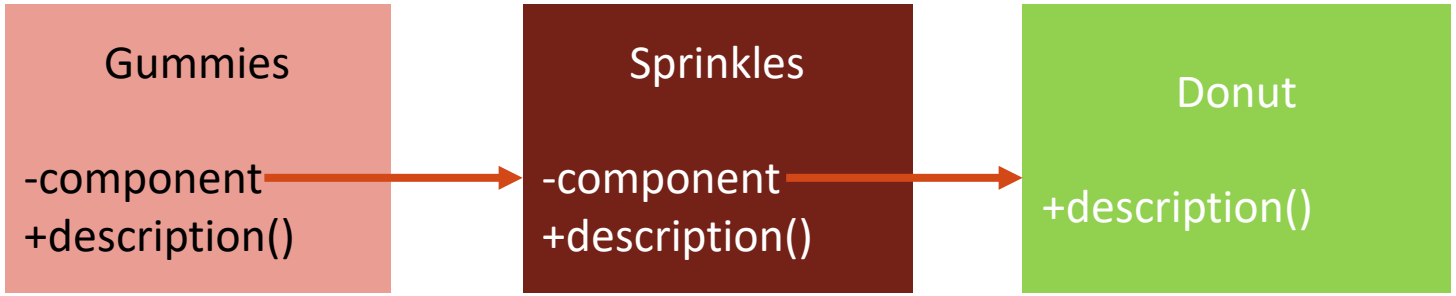
Introduction - Decorator

- We'll "decorate" our Donut object with toppings
  - Wrap our **donut** object with **Sprinkles** topping



Introduction - Decorator

- We'll "decorate" our Donut object with toppings
  - Wrap our **donut** object with **Sprinkles** topping
  - Wrap that with another topping, **Gummies**



Defining Decoration

We ‘wrap’ our base object using decorative pieces.

In the donut analogy, it could be gummies, sprinkles, or even the filling (think of this as decorating the inside).

These decorative pieces

- Re-use the code of the Base Object also known as the **Base Component**
- Extend the functionality while maintaining the same interface by **adding specialized behaviours** or overriding attributes.

> **Decorator** is a structural design pattern that lets you **attach new behaviors** to objects by placing these objects inside special wrapper objects that contain the behaviors.

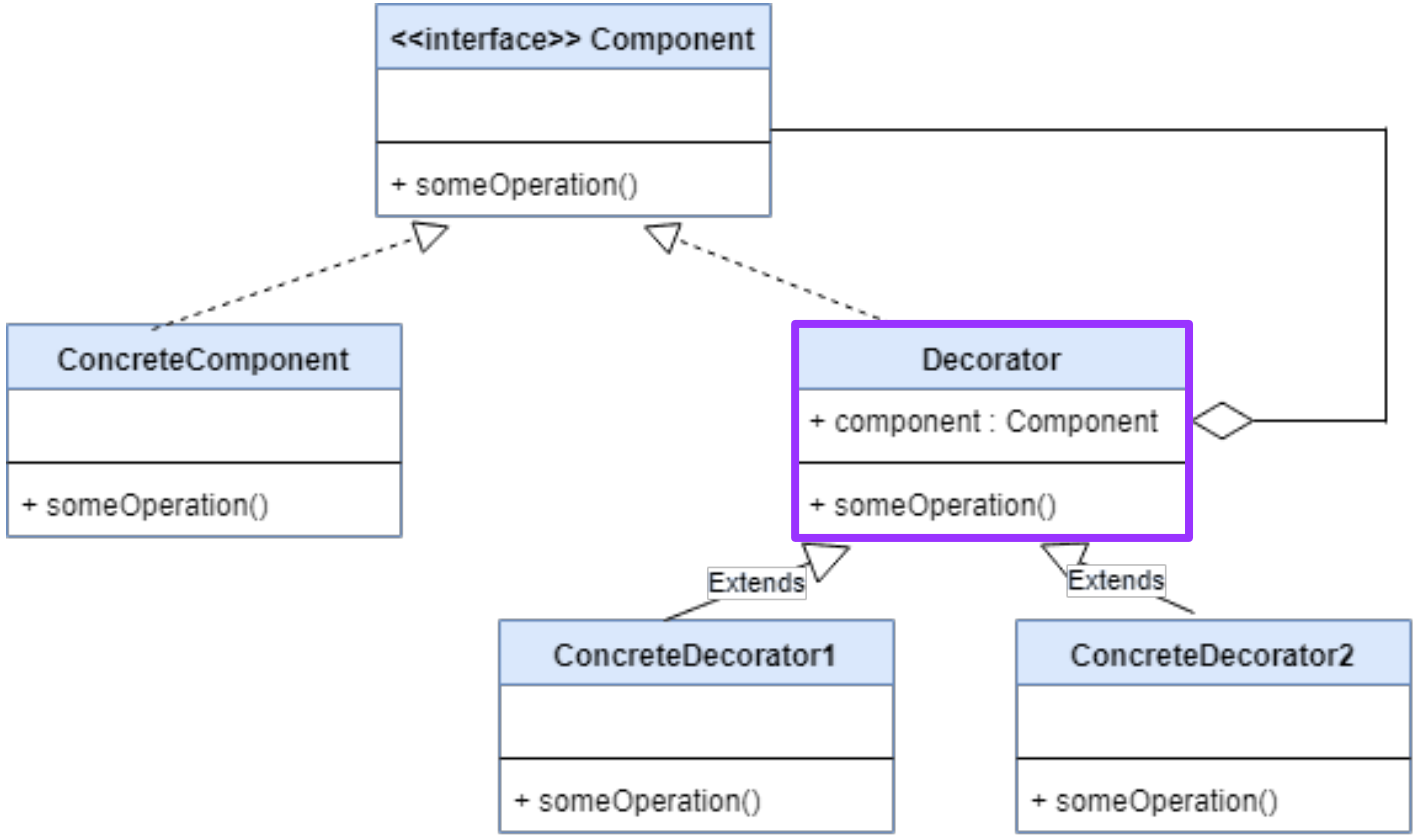
Decorators

Decorators are just **wrappers** with **one added feature.**

All decorators inherit from a **Base Decorator Class**. This allows us to define multiple different wrappers that can wrap around the concrete component and around other wrappers as well.

Take a look at the UML Diagram.

The **Decorator** is a **base class** that **wraps around** anything that implements a **Component interface**, like ConcreteComponent for example.



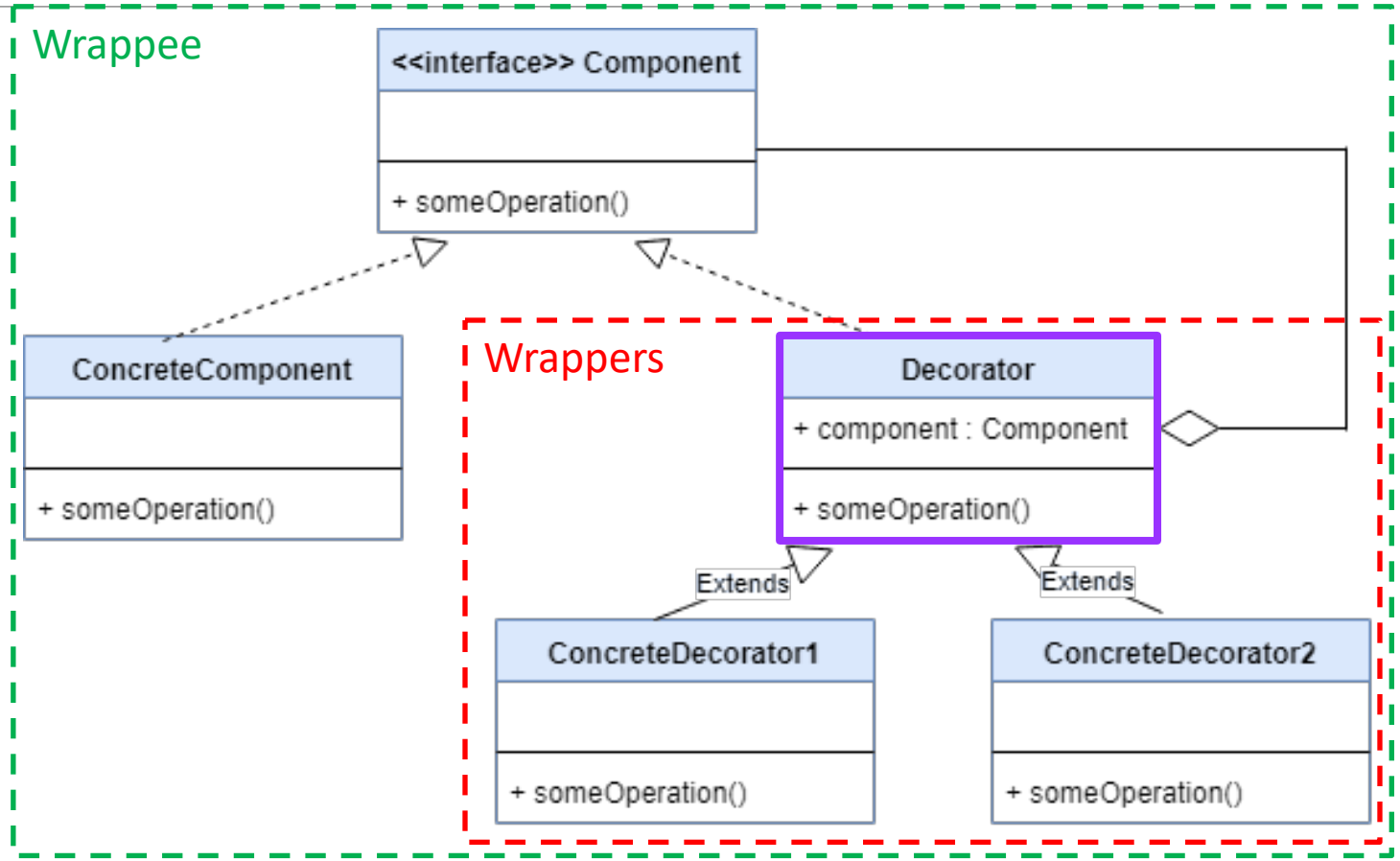
Decorators

Decorators are just wrappers with one added feature.

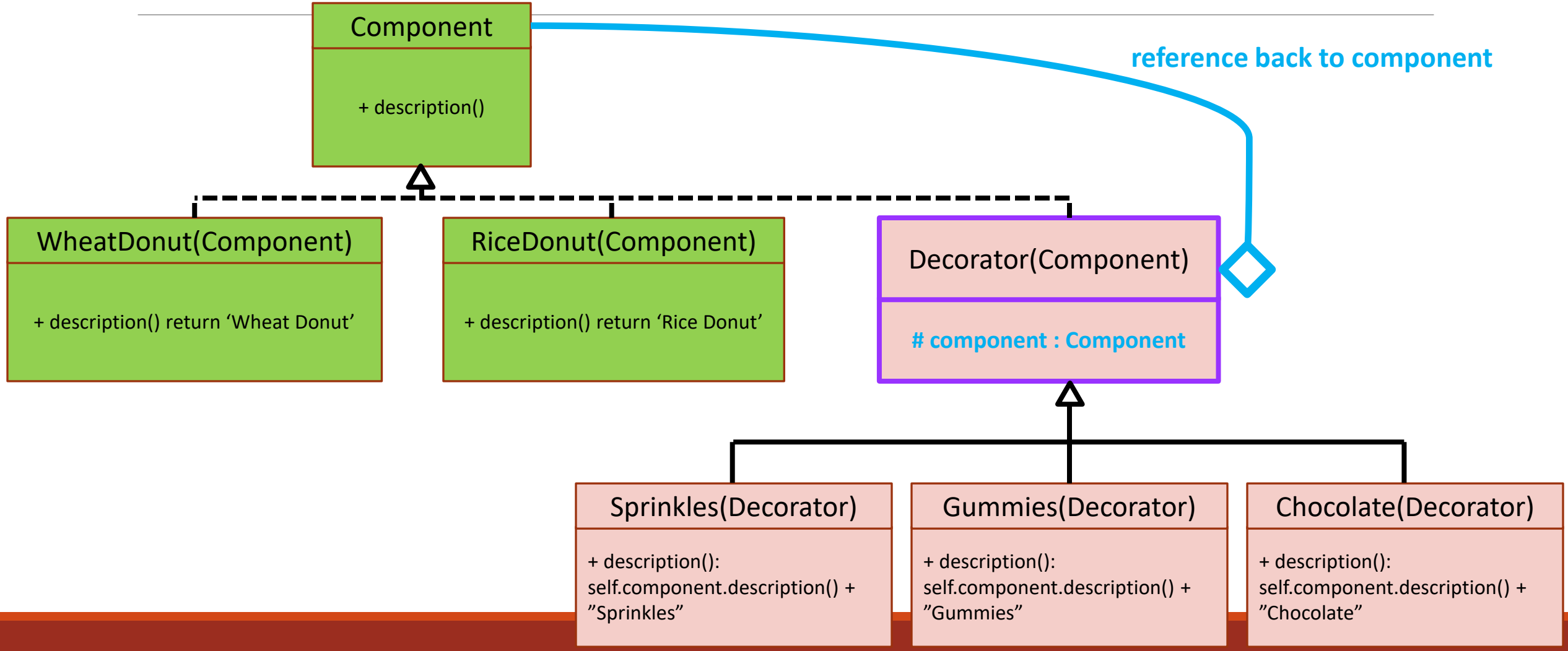
All decorators inherit from a **Base Decorator Class**. This allows us to define multiple different wrappers that can wrap around the concrete component and around other wrappers as well.

Take a look at the UML Diagram.

The **Decorator** is a **base class** that **wraps around** anything that implements a **Component interface**, like ConcreteComponent for example.

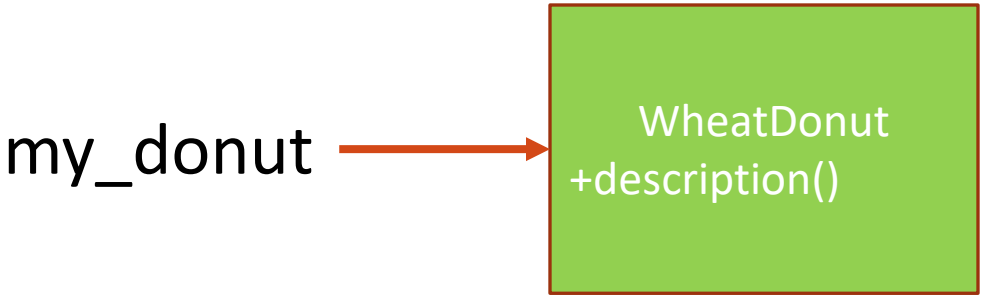


Donuts…


Component
+ description()
reference back to component
WheatDonut(Component)
+ description() return 'Wheat Donut'
RiceDonut(Component)
+ description() return 'Rice Donut'
Decorator(Component)
# component : Component
Sprinkles(Decorator)
+ description():
self.component.description() +
"Sprinkles"
Gummies(Decorator)
+ description():
self.component.description() +
"Gummies"
Chocolate(Decorator)
+ description():
self.component.description() +
"Chocolate"

Introduction - Decorator

- my_donut = **WheatDonut**()



- Create **WheatDonut** component

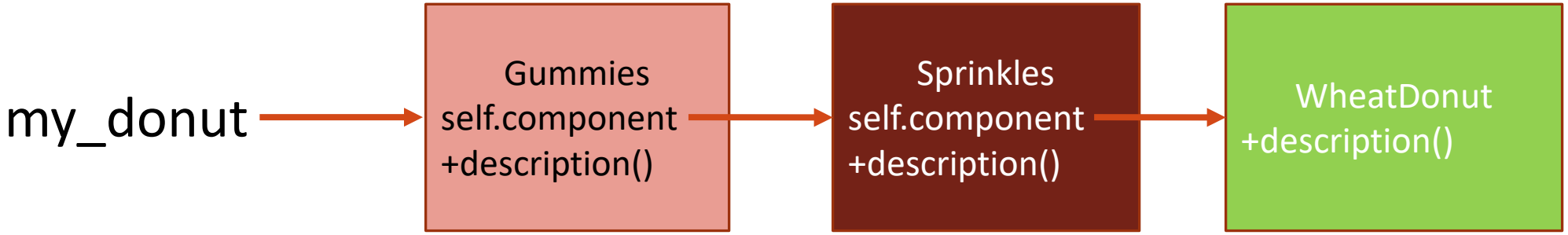
Introduction - Decorator

- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)

- Create **Sprinkles** object, set its self.component to point to my_donut
- **Sprinkles'** grandparent base class is Component, so the reference it returns is saved into my_donut

Introduction - Decorator

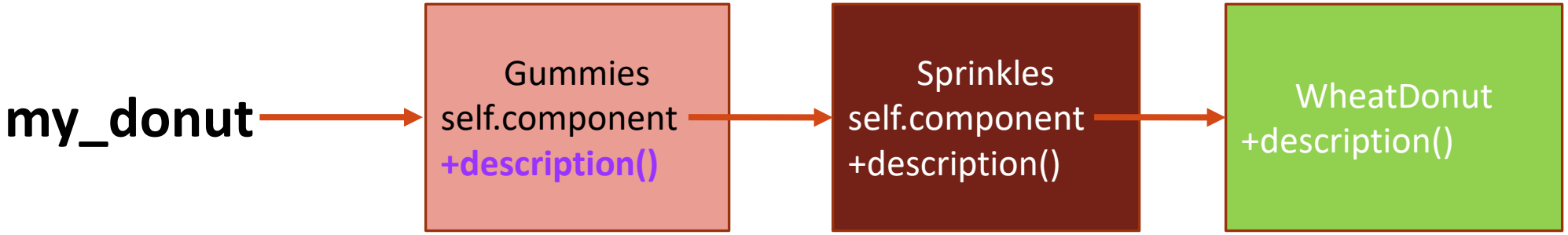
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)



- Create **Gummies** object, set its self.component to point to my_donut
- **Gummies**' grandparent base class is Component, so the reference it returns is saved into my_donut

Introduction - Decorator

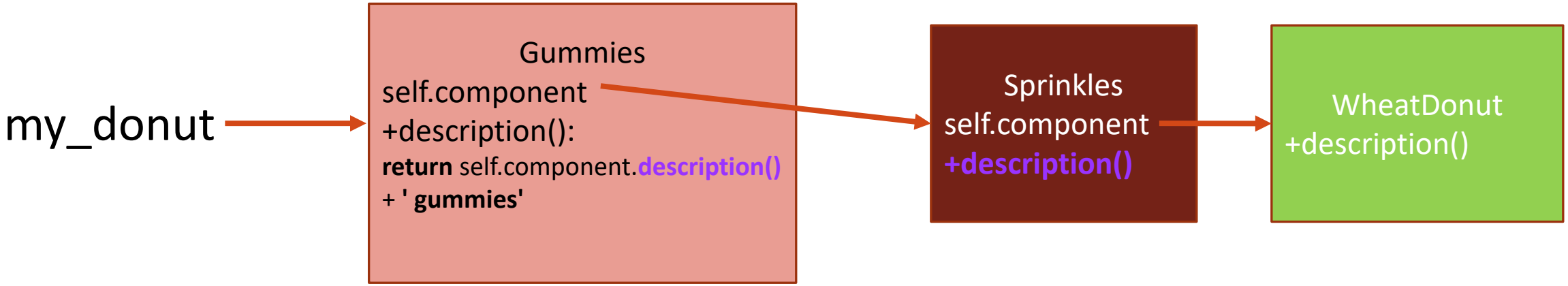
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(**my_donut**.**description()**)



- Notice how it looks like a **linked list** starting from **my_donut**
- Get total description of the donut by calling **my_donut**.description();

Introduction - Decorator

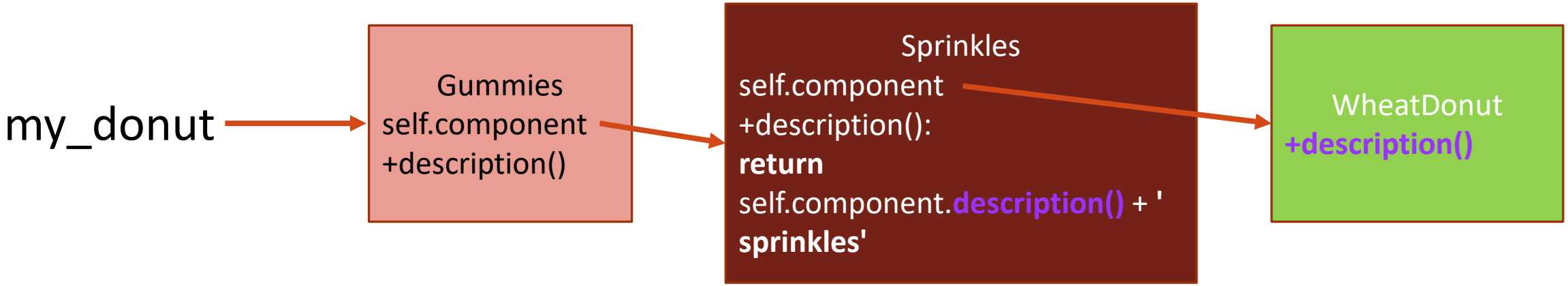
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.**description**())



- Calls to description method will chain into toppings until they reach **WheatDonut**
- Gummies' description method calls Sprinkles' description function

Introduction - Decorator

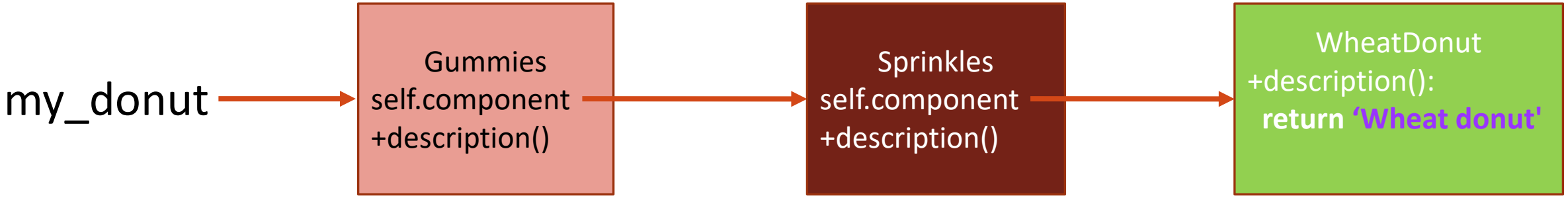
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- Sprinkles' description method calls **WheatDonut's** description function

Introduction - Decorator

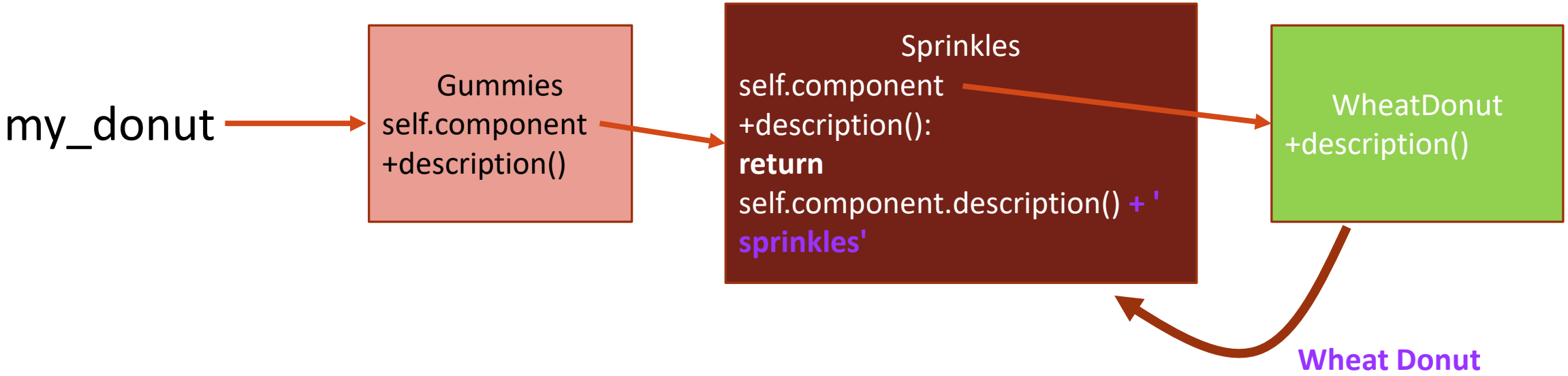
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- String is appended as we return from **WheatDonut's** description() method calls
- **WheatDonut** **description**() method returns "**Wheat donut**"

Introduction - Decorator

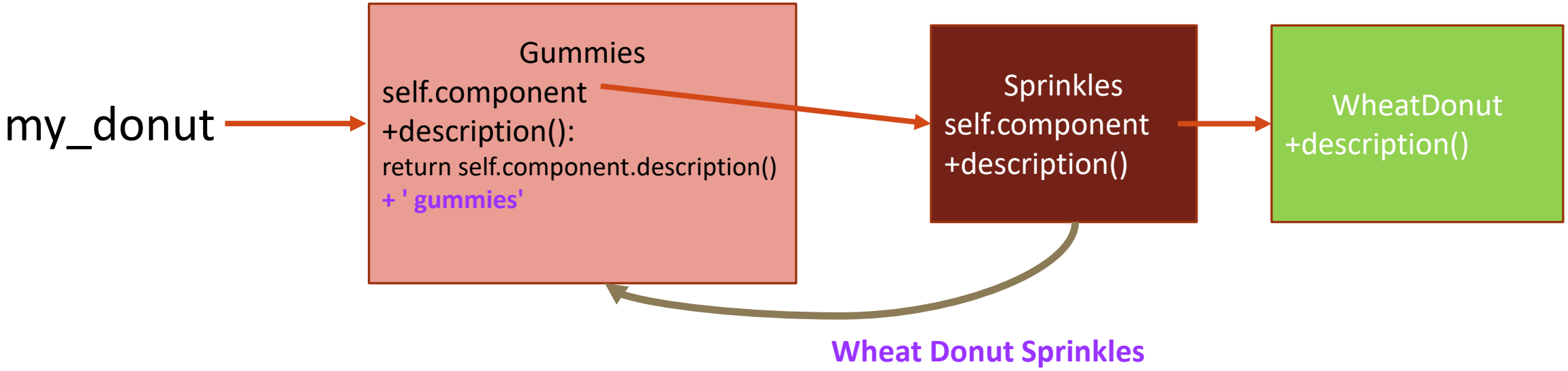
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- "**sprinkles**" appended to returned "**Wheat donut**". This string also returned

Introduction - Decorator

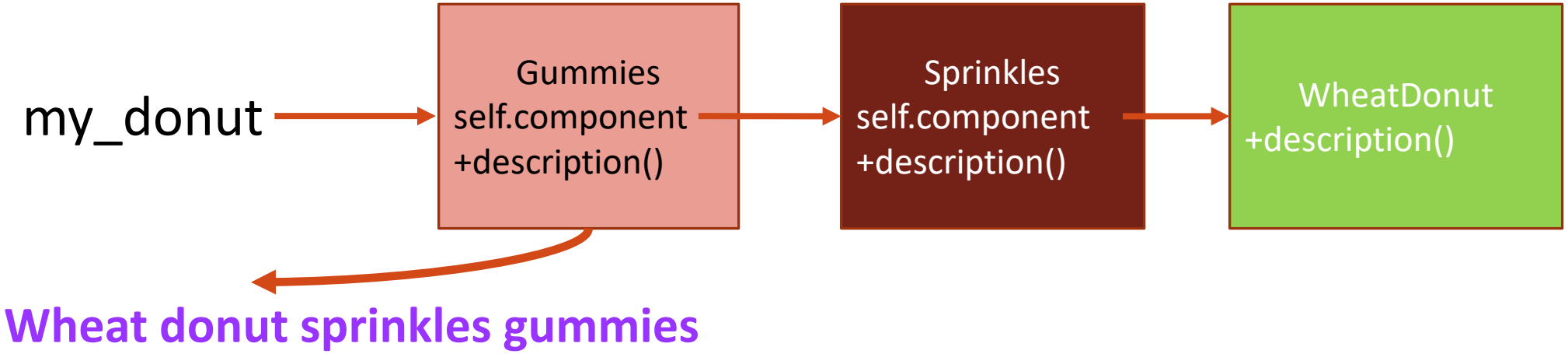
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- "**gummies**" appended to returned "**Wheat donut sprinkles**". This string also returned

Introduction - Decorator

- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())

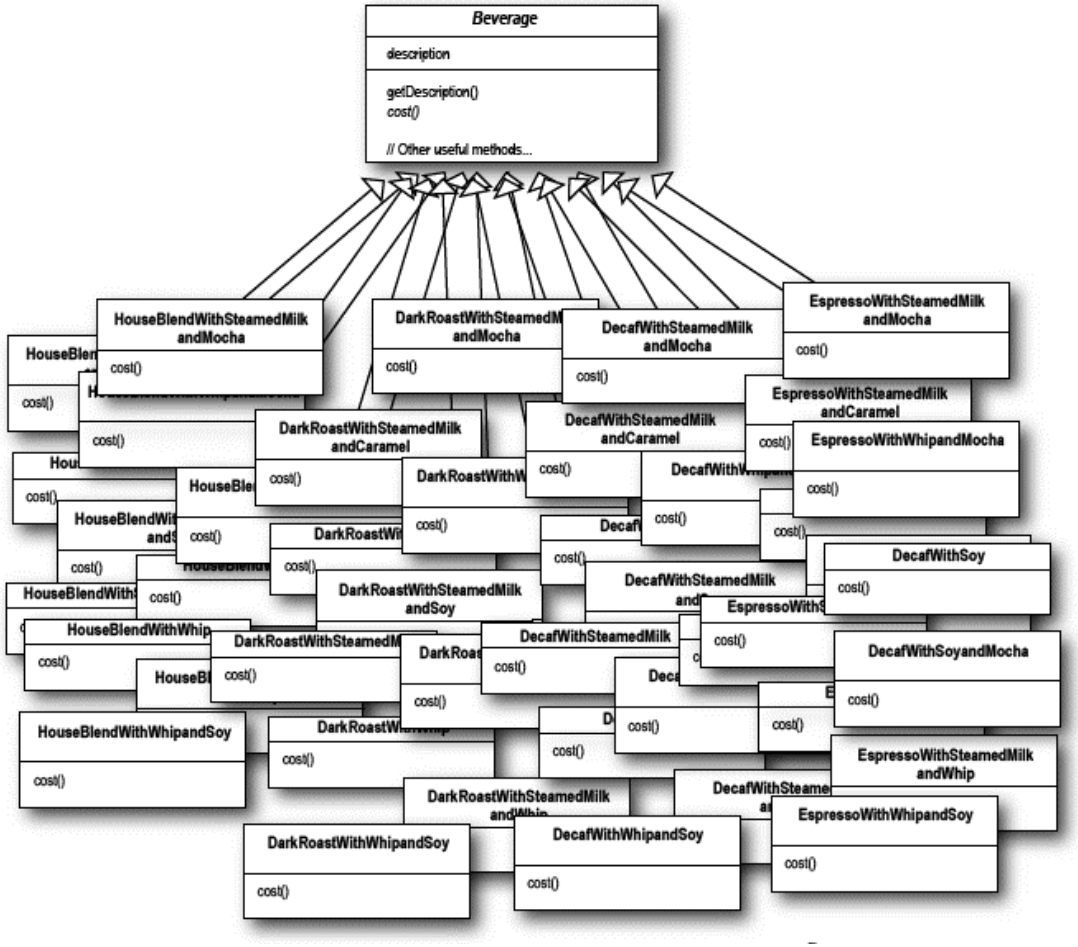


- Final "**Wheat donut sprinkles gummies**" returned to print out

simple_decorator.py

Decorator Advantages

We get to avoid this:



Decorator Advantages

- Adhere to the Single Responsibility Principle. Each **decorator is responsible for one behavior**.
- Adhere to the Open Close Principle. Our Concrete Component is closed to modifications but open to extension (Indirectly via decorators).
- Allow for **multiple combinations and behaviours without the need for multiple inheritance**. We avoid directly sub-classing from the concrete component itself.
- Add or remove behaviors at run-time.

Decorator Disadvantages

- Can be **difficult to remove or access the concrete object or a specific wrapper** in the stack of wrappers.
- Subclasses need to maintain the same interface (probably a good thing, but restrictive).
- Can be **difficult to trace and debug if too many decorators** are layered one upon the other.
- Hard to avoid scenarios where the **behavior changes based on the order** in the decorators stack.

Recap

1

The **Component** declares the common interface for both wrappers and wrapped objects.

2

Concrete Component is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.

3

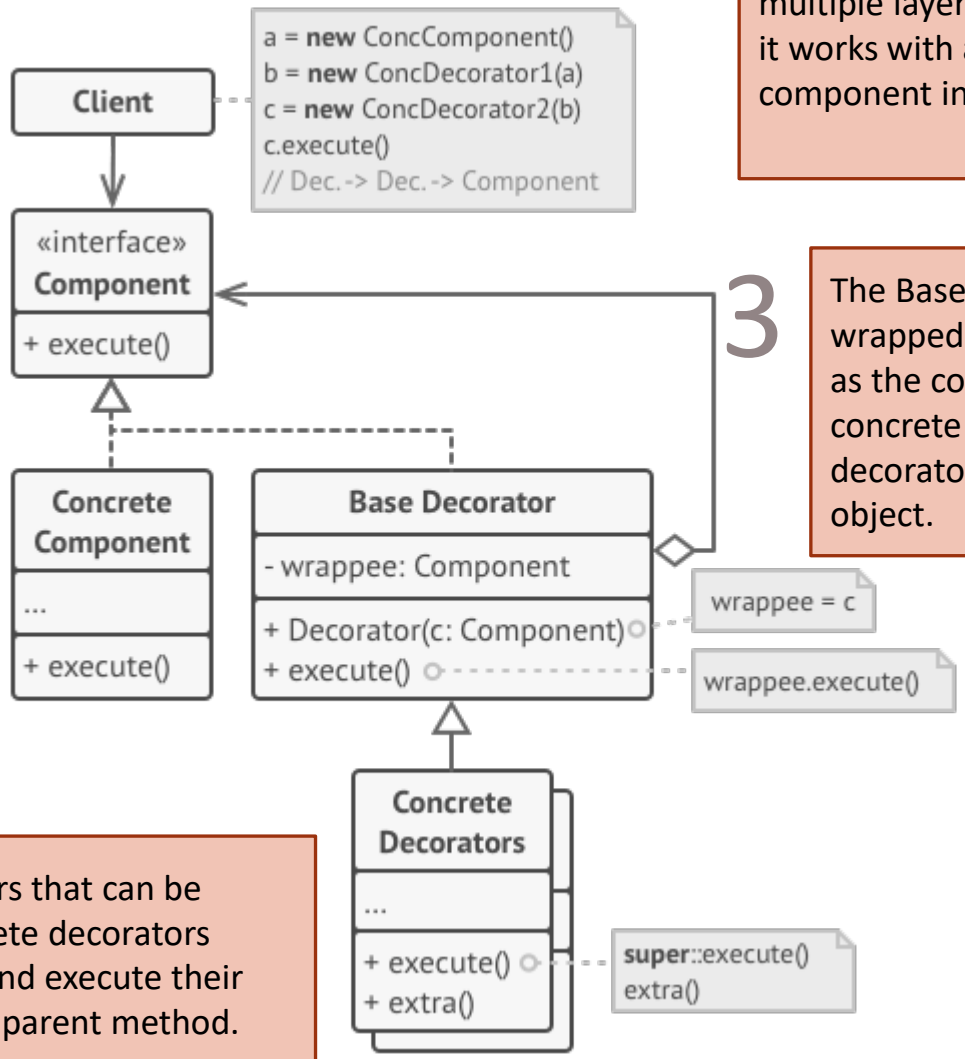
The Base Decorator class has a field for referencing a wrapped object. The field's type should be declared as the component interface so it can contain both concrete components and decorators. The base decorator delegates all operations to the wrapped object.

4

Concrete Decorators define extra behaviors that can be added to components dynamically. Concrete decorators override methods of the base decorator and execute their behavior either before or after calling the parent method.

5

The Client can wrap components in multiple layers of decorators, as long as it works with all objects via the component interface.



Bonus Decorator Pattern

Say we have a FileDataSource.

We can Read and Write to this Data Source.

Say we wanted to **encrypt** the data before writing and decrypt it after reading

Say we also wanted to **compress** the data using some compression algorithm before writing and decompress it before reading.

Now what if we wanted to **encrypt and compress** the data?

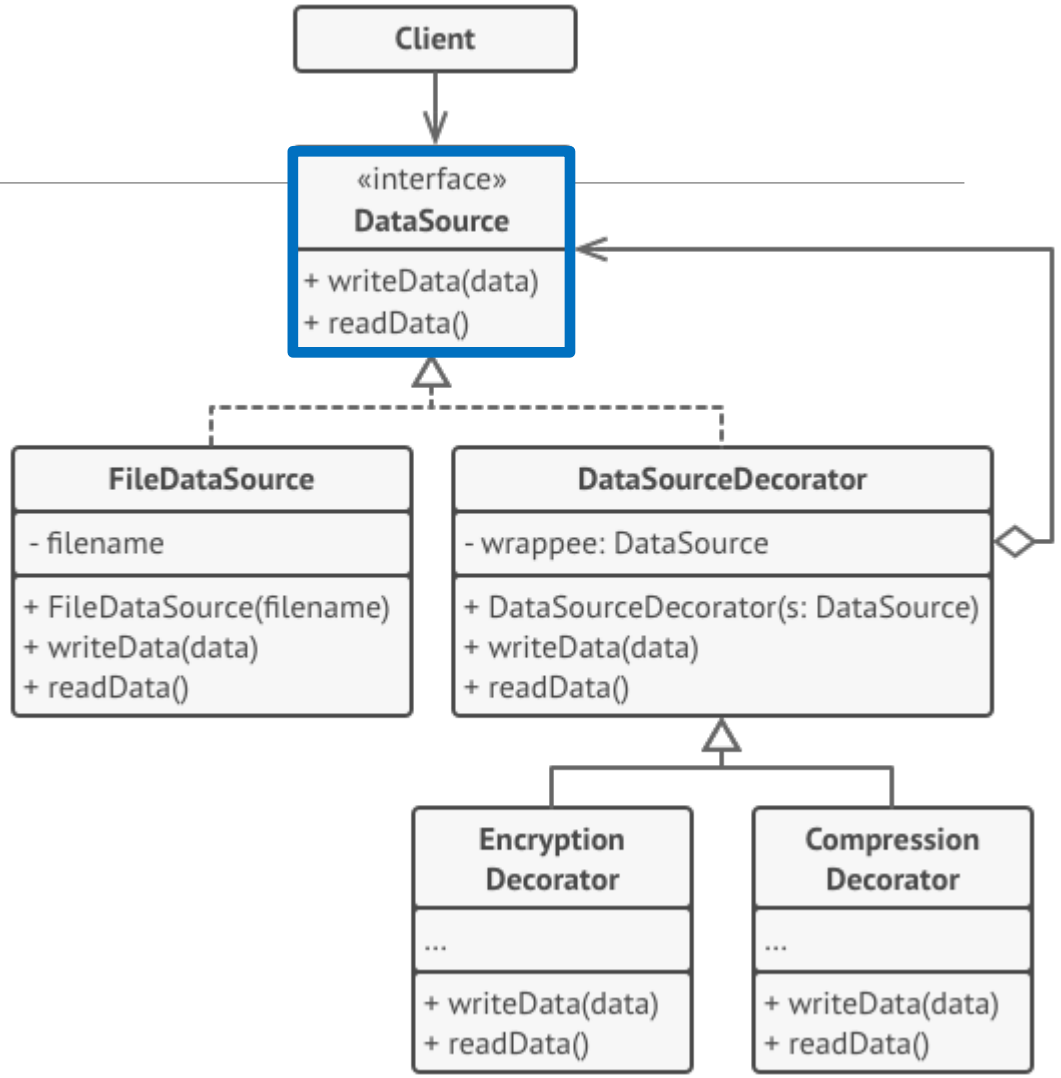
Let's avoid a multiple inheritance scenario (CompressedEncryptedFileDataSource – Yikes) .

Let's implement wrappers that extend these behaviours and see it in action!

Data Source

Here is the UML Diagram for the DataSource implementation.

Remember these are language agnostic. They don't depend on any specific programming language

In Python we have the flexibility to not create a separate formal **DataSource** Interface and just leave it to Duck Typing.

decorator_example.py